



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS QUIXADÁ
CURSO DE GRADUAÇÃO EM SISTEMAS DE INFORMAÇÃO

TAILAN DE SOUZA OLIVEIRA

**DECOMPOSIÇÃO DO MAX-HRT PARA ALOCAÇÃO DO SISU COM INDIFERENÇA
DE MODALIDADES USANDO AGRUPAMENTO DE CURSOS**

QUIXADÁ
2026

TAILAN DE SOUZA OLIVEIRA

DECOMPOSIÇÃO DO MAX-HRT PARA ALOCAÇÃO DO SISU COM INDIFERENÇA DE
MODALIDADES USANDO AGRUPAMENTO DE CURSOS

Projeto de Pesquisa apresentado ao Curso de Graduação em Sistemas de Informação do Campus Quixadá da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Sistemas de Informação.

Orientador: Prof. Dr. Críston Pereira de Souza.

QUIXADÁ

2026

AGRADECIMENTOS

Agradeço, primeiramente, ao Prof. Dr. Críston Pereira de Souza pela excelente orientação, pelo incentivo constante e pelas inúmeras discussões em sala, que foram fundamentais para definir a melhor abordagem para o desenvolvimento desta pesquisa. Agradeço à Andressa Lima Colares pela valiosa colaboração ao longo desta pesquisa, especialmente na implementação da formulação, na compreensão do modelo e nas discussões sobre a melhor forma de aplicar o *MAX-HRT* aos casos de teste. Sua contribuição também foi essencial para o desenvolvimento das heurísticas LCF e ODF. Agradeço, ainda, ao Prof. Dr. Davi Romero de Vasconcelos e a Lucas Maurício Braga pela disponibilização dos dados de entrada utilizados nesta pesquisa, os quais foram indispensáveis para a realização dos experimentos.

SUMÁRIO

1	INTRODUÇÃO	6
1.1	Objetivos	7
<i>1.1.1</i>	<i>Objetivo Geral</i>	<i>8</i>
<i>1.1.2</i>	<i>Objetivos Específicos</i>	<i>8</i>
1.2	Organização do trabalho	8
2	FUNDAMENTAÇÃO TEÓRICA	9
2.1	O Sistema de Seleção Unificada (SISU)	9
<i>2.1.1</i>	<i>CrITÉRIOS de Classificação e Desempate</i>	<i>9</i>
<i>2.1.2</i>	<i>CrITÉRIOS de Desempate</i>	<i>10</i>
<i>2.1.3</i>	<i>Escolha Estratégica e Notas de Corte</i>	<i>10</i>
<i>2.1.4</i>	<i>Modalidades de Concorrência e a Lei de Cotas</i>	<i>10</i>
<i>2.1.5</i>	<i>Classificação Hierárquica e Remanejamento Automático</i>	<i>11</i>
<i>2.1.6</i>	<i>Vagas Remanescentes e Maximização da Ocupação</i>	<i>12</i>
2.2	Problemas de Matching e Alocação Educacional	12
<i>2.2.1</i>	<i>Stable Matching e o Algoritmo de Gale-Shapley</i>	<i>13</i>
<i>2.2.2</i>	<i>Formalização do Algoritmo de Gale-Shapley</i>	<i>14</i>
<i>2.2.3</i>	<i>Exemplo de execução Algoritmo de Gale-Shapley</i>	<i>15</i>
2.3	O Problema MAX-HRT	16
<i>2.3.1</i>	<i>Tipos de Estabilidade no Problema de Emparelhamento</i>	<i>17</i>
<i>2.3.2</i>	<i>Formulação por Programação Inteira</i>	<i>18</i>
2.4	Teoria de Grafos e Clusterização de Cursos	19
<i>2.4.1</i>	<i>Conceitos Básicos de Grafos</i>	<i>19</i>
<i>2.4.2</i>	<i>Particionamento e Cortes em Grafos</i>	<i>21</i>
<i>2.4.3</i>	<i>Clusterização e Métodos Baseados em Min-cut</i>	<i>22</i>
<i>2.4.4</i>	<i>Modelagem do SISU como Grafo Curso–Curso</i>	<i>22</i>
<i>2.4.5</i>	<i>Candidatos Internos, Candidatos de Fronteira e Impacto no MAX-HRT</i>	<i>22</i>
2.5	Algoritmo Proposto de Decomposição e Mecanismo de Migração	23
<i>2.5.1</i>	<i>Clusterização de Cursos via K-Means Constrained</i>	<i>23</i>
<i>2.5.2</i>	<i>Algoritmo de Decomposição e Migração de Candidatos de Fronteira</i>	<i>24</i>
<i>2.5.3</i>	<i>Algoritmos Incrementais de Matching</i>	<i>24</i>

2.5.4	<i>ASISU</i>	25
2.5.5	<i>LCF (Least Competitive First)</i>	26
2.5.6	<i>ODF (Observed Deficit First)</i>	26
2.5.7	<i>Exemplo Ilustrativo de Formação de Clusters</i>	27
2.5.8	<i>Mecanismo de Migração e Convergência</i>	28
3	TRABALHOS RELACIONADOS	29
3.1	<i>College admissions and the stability of marriage</i>	29
3.2	<i>The Hospitals/Residents Problem with Ties</i>	29
3.3	<i>An Integer Programming Approach to the Hospitals/Residents Problem with Ties</i>	30
3.4	<i>SISU Allocation with Indifference among Admission Modalities: a MAX-HRT Model and Comparison of Tie-Breaking Criteria</i>	31
3.5	<i>Multi-Level Partitioning and Distribution of the Assignment Problem for Large-Scale Multi-Robot Task Allocation</i>	31
3.6	<i>Análise comparativa</i>	33
4	METODOLOGIA	34
4.1	<i>Construção do Dataset Experimental</i>	34
4.2	<i>Modelagem do Problema MAX-HRT</i>	35
4.3	<i>Estratégia de Decomposição e Clusterização</i>	36
4.3.1	<i>K-Means Constrained</i>	36
4.3.2	<i>Particionamento via Biblioteca METIS</i>	36
4.3.3	<i>Escolha do Número de Clusters</i>	37
4.4	<i>Alocação na sub-instância por Programação Inteira</i>	37
4.5	<i>Tratamento de Candidatos de Fronteira e emparelhamento incremental</i>	38
4.6	<i>Implementação e Ferramentas Utilizadas</i>	38
4.6.1	<i>Base de Dados do SISU</i>	38
4.6.2	<i>Ambiente Computacional e Ferramentas</i>	39
4.7	<i>Avaliação Experimental</i>	39
4.7.1	<i>Algoritmos Avaliados</i>	39
4.7.2	<i>ASISU — Baseline</i>	40
4.7.3	<i>Least Competitive First (LCF)</i>	40
4.7.4	<i>Observed Deficit First (ODF)</i>	40

4.7.5	<i>IP+GS Incremental — Algoritmo Proposto</i>	40
4.7.6	<i>Métricas de Avaliação</i>	41
4.7.7	<i>Artefatos Gerados</i>	41
4.8	Cronograma	42
5	RESULTADOS PRELIMINARES	43
5.1	Análise Exploratória dos Dados	43
5.1.1	<i>Análise de Hubs e Centralidade da Rede de Concorrência</i>	43
5.1.2	<i>Cursos e suas principais conexões</i>	45
5.1.3	<i>Análise de Candidatos de Fronteira com e sem Medicina via Metis</i>	47
5.2	Análise de Demanda, Oferta e Clusterização via k-means-constrained .	48
5.3	Comparação entre K-Means Constrained e METIS	49
5.4	Desempenho Geral do Modelo MAX-HRT	51
5.4.1	<i>Análise dos Resultados Computacionais</i>	52
5.5	Discussão e Viabilidade Computacional para o SISU	52
	REFERÊNCIAS	54

1 INTRODUÇÃO

O Sistema de Seleção Unificada (SISU) é uma das principais formas de acesso ao ensino superior público no Brasil. Seu processo de alocação incorpora regras de reserva de vagas estabelecidas pela Lei de Cotas (Lei nº 12.711/2012), recentemente atualizada pela Lei nº 14.723/2023 (BRASIL, 2023b). Neste trabalho, considera-se a regra de classificação em que todos os candidatos concorrem inicialmente na modalidade de Ampla Concorrência (AC), inclusive aqueles elegíveis às políticas afirmativas, sendo avaliados nas modalidades de reserva apenas quando não classificados nessa etapa. Essa legislação define a reserva de vagas para diferentes grupos socioeconômicos, tornando o processo de alocação de candidatos complexo do ponto de vista combinatório e computacional.

Do ponto de vista computacional, o problema pode ser modelado como uma variante do *Hospitals/Residents Problem with Ties* (HRT) (Kwanashie; Manlove, 2014), derivado do modelo clássico de emparelhamento estável de *Gale-Shapley* (1962). Nessa modelagem, os candidatos correspondem aos residentes, enquanto os pares curso–modalidade de concorrência correspondem aos hospitais, cada um com uma capacidade específica. A indiferença surge porque, para um candidato, diferentes modalidades de um mesmo curso podem ser consideradas equivalentes quanto ao resultado final de ingresso. Assim, a presença de empates nas listas de preferência pode produzir emparelhamentos estáveis de cardinalidades distintas. O problema de encontrar um emparelhamento estável de cardinalidade máxima, denominado *MAX-HRT*, é NP-difícil (Kwanashie; Manlove, 2014), o que torna a resolução exata inviável na escala nacional do SISU.

Para contornar essa intratabilidade, este trabalho propõe decompor o problema em subinstâncias menores por meio do agrupamento de cursos em *clusters* balanceados. A similaridade entre dois cursos é definida pela quantidade de candidatos inscritos simultaneamente em ambos, de modo que cursos com alta co-inscrição tendem a pertencer ao mesmo agrupamento. São investigadas e comparadas duas estratégias de particionamento. A primeira utiliza o algoritmo *K-Means Constrained* (Levy-Kramer, 2018), adotado por permitir a formação de *clusters* com tamanhos equilibrados a partir da matriz de co-inscrição. A segunda utiliza a biblioteca METIS (Karypis; Kumar, 1998), que implementa um algoritmo multinível de particionamento de grafos. Nesse caso, a co-inscrição entre cursos é representada como um grafo ponderado ($G = (V, E, W)$), e o particionamento busca minimizar o corte entre os grupos, reduzindo o número de candidatos de fronteira, isto é, candidatos cujas duas opções pertencem a *clusters* distintos. Cada *cluster*

é então tratado como uma subinstância independente de *MAX-HRT*, resolvida pelo modelo de Programação Inteira proposto por (Kwanashie; Manlove, 2014).

Candidatos inscritos em cursos pertencentes a *clusters* distintos, denominados candidatos de fronteira, são inicialmente associados ao *cluster* do curso de primeira opção. Caso sejam rejeitados em todos os pares (curso, modalidade) desse *cluster*, esses candidatos migram para o *cluster* correspondente à segunda opção. Nesse *cluster*, executa-se um procedimento incremental baseado no algoritmo de *Gale-Shapley* apenas sobre as vagas afetadas. O processo se repete até que não haja novas migrações. Como cada candidato possui no máximo duas opções de curso, o mecanismo impede ciclos de migração entre *clusters*.

Para avaliar a qualidade das soluções obtidas pela decomposição proposta, este trabalho utiliza como referência experimental os dados e resultados do SISU 2023 apresentados por (Oliveira *et al.*, 2026). Esse estudo, submetido ao SBPO 2026, investiga diferentes critérios de desempate para o *MAX-HRT* aplicado ao SISU. Em particular, são avaliados três algoritmos baseados em *Gale-Shapley* ASISU, *Least Competitive First* (LCF) e *Observed Deficit First* (ODF). O ASISU reproduz a ordem canônica definida no edital e é utilizado como *baseline*. O LCF prioriza vagas com menor razão entre candidatos elegíveis e vagas disponíveis. O ODF, por sua vez, reordena as propostas dos candidatos com base no déficit de vagas observado após uma primeira execução do SISU. Nos experimentos com dados reais, o LCF obteve o maior número de alocações entre as estratégias avaliadas.

A decomposição em *clusters* visa tornar o modelo de Programação Inteira computacionalmente viável. Enquanto a instância nacional do SISU envolve mais de um milhão de inscrições, cada subinstância gerada pelo agrupamento contém apenas algumas centenas de candidatos e dezenas de pares curso–modalidade, faixa em que a modelagem *MAX-HRT* tende a convergir mais rapidamente para uma solução ótima (Kwanashie; Manlove, 2014). A qualidade global da solução é avaliada pelas mesmas métricas adotadas por (Oliveira *et al.*, 2026): taxa de ocupação de vagas, número de instabilidades e tempo de execução.

1.1 Objetivos

Esta seção apresenta os objetivos deste trabalho, com ênfase na decomposição do problema de alocação do SISU em subproblemas menores, visando maior eficiência computacional.

1.1.1 *Objetivo Geral*

Desenvolver e avaliar um método heurístico de decomposição do problema *MAX-HRT* aplicado à alocação do SISU, utilizando agrupamento de cursos para reduzir o tempo de processamento em relação a abordagens exatas e buscar melhores taxas de ocupação de vagas quando comparado às heurísticas avaliadas.

1.1.2 *Objetivos Específicos*

- Projetar heurística para o *MAX-HRT* da alocação do SISU com indiferença entre modalidades, baseada na decomposição do grafo de cursos, execução do modelo MIP do *MAX-HRT* em cada *cluster*, e tratamento dos candidatos na fronteira entre *clusters* para atingir solução estável.
- Avaliar a heurística proposta utilizando dados abertos do SISU, e comparação com heurísticas baseadas em quebra de empates e execução do *Gale-Shapley*.
- Avaliar o impacto do número de clusters no número de candidatos alocados.

1.2 **Organização do trabalho**

Este trabalho está organizado da seguinte forma. O Capítulo 2 apresenta a fundamentação teórica, cobrindo o funcionamento do SISU, a modelagem como *MAX-HRT*, a formulação por Programação Inteira e o algoritmo proposto de decomposição por *clusters* usando *K-Means Constrained* ou grafos. O Capítulo 3 discute os trabalhos relacionados. O Capítulo 4 descreve a metodologia, incluindo a construção do *dataset*, a estratégia de clusterização, o tratamento dos candidatos de fronteira e a avaliação experimental. O Capítulo 5 apresenta os resultados preliminares obtidos com os dados do SISU 2023.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne a base teórica utilizada na pesquisa. A Seção 2.1 apresenta o funcionamento do SISU, suas regras, cotas e mecanismos de remanejamento. A Seção 2.2 introduz problemas de emparelhamento e o algoritmo de Gale–Shapley. A Seção 2.3 formaliza o problema *MAX-HRT* e sua formulação por Programação Inteira. A Seção 2.4 discute grafos, cortes e a modelagem curso–curso do SISU. Por fim, a Seção 2.5 apresenta a decomposição por *clusters*, o tratamento de candidatos de fronteira e as heurísticas incrementais usadas neste trabalho.

2.1 O Sistema de Seleção Unificada (SISU)

O SISU é uma das principais formas de ingresso no ensino superior público no Brasil, operando como um sistema centralizado de alocação de candidatos com base nas notas do Exame Nacional do Ensino Médio (ENEM) (BRASIL, 2025b). Seu funcionamento é regulamentado por regras de classificação, modalidades de concorrência, políticas de cotas e critérios de desempate definidos pelo Ministério da Educação (Brasil, 2012; BRASIL, 2023b).

2.1.1 Critérios de Classificação e Desempate

A classificação dos candidatos é baseada na nota final obtida no ENEM, que pode ser calculada por média aritmética simples ou por média ponderada, conforme os critérios definidos pelo curso e pela instituição.

$$M_s = \frac{N_{LC} + N_{CH} + N_{CN} + N_{MT} + N_{RED}}{5} \quad (2.1)$$

Na média aritmética ponderada, cada área recebe um peso w_i definido pela instituição:

$$M_p = \frac{\sum_{i=1}^5 (N_i \cdot w_i)}{\sum_{i=1}^5 w_i} \quad (2.2)$$

onde N_i representa a nota na i -ésima área do conhecimento Linguagens e Códigos (N_{LC}), Ciências Humanas (N_{CH}), Ciências da Natureza (N_{CN}), Matemática (N_{MT}) e Redação (N_{RED}) e w_i o peso atribuído pela instituição a cada área.

2.1.2 *CrITÉrios de Desempate*

Em situações de igualdade entre notas finais, o SISU aplica critérios de desempate baseados na hierarquia das áreas do ENEM. Em geral, a ordem de prioridade considera Redação, Linguagens, Matemática, Ciências da Natureza e Ciências Humanas (BRASIL, 2025b), produzindo uma ordenação estrita entre candidatos dentro de cada modalidade de concorrência.

Embora esses critérios reduzam ou eliminem empates entre candidatos nas listas de preferência dos cursos, a existência de múltiplas modalidades de concorrência para um mesmo curso introduz uma forma distinta de indiferença. Neste trabalho, assume-se que candidatos interessados apenas no curso e na instituição são indiferentes entre as modalidades de ingresso associadas ao mesmo curso. Essa estrutura de indiferença motiva a modelagem do problema como uma instância do *Hospitals/Residents with Ties* (HRT), discutida na Seção 2.3.

2.1.3 *Escolha Estratégica e Notas de Corte*

Durante o período de inscrição, cada candidato pode selecionar até duas opções de curso. O sistema realiza atualizações periódicas da nota de corte a menor nota entre os candidatos classificados dentro das vagas disponíveis, que é dinâmica e varia à medida que novos candidatos se inscrevem (BRASIL, 2026b).

Esse mecanismo pode induzir comportamento estratégico, pois as escolhas iniciais dos candidatos nem sempre refletem a melhor alocação possível ao final do processo. Além disso, a limitação a duas opções de curso pode contribuir para a ociosidade de vagas. Do ponto de vista da modelagem, este trabalho trata esse cenário por meio da formulação *MAX-HRT*, discutida na Seção 2.3.

2.1.4 *Modalidades de Concorrência e a Lei de Cotas*

O SISU estrutura as vagas em três grupos de modalidades, conforme as diretrizes do Ministério da Educação (BRASIL, 2025c):

- Ampla Concorrência (AC): Modalidade padrão aberta a todos. Todos os candidatos disputam inicialmente estas vagas. Candidatos cotistas também entram nesta primeira listagem. Se não forem selecionados, passam a disputar as vagas de suas respectivas cotas.
- Lei de Cotas (Lei n.º 12.711/2012): destinadas a candidatos egressos integralmente do ensino médio público, subdivididas em dois grupos:

- Independente de renda (LI): inclui as subcategorias LI_EP (escola pública geral), LI_PCD (pessoas com deficiência), LI_Q (quilombolas) e LI_PPI (pretos, pardos ou indígenas);
- Baixa renda (LB): exige renda familiar per capita igual ou inferior a um salário mínimo, com subcategorias análogas (LB_EP, LB_PCD, LB_Q e LB_PPI).
- Ações Afirmativas Próprias: ofertadas opcionalmente pelas instituições, de forma independente da Lei de Cotas, podendo assumir a forma de bônus percentual na nota (B) ou reserva de vagas para grupos específicos (V).

2.1.5 Classificação Hierárquica e Remanejamento Automático

A classificação no SISU ocorre sob o formato de concorrência concomitante, priorizando o mérito acadêmico antes da aplicação das reservas de vagas (BRASIL, 2023a). Inicialmente, todos os inscritos concorrem às vagas de Ampla Concorrência (AC), independentemente de cumprirem os requisitos para modalidades de cotas. Caso o candidato não obtenha classificação suficiente nessa modalidade geral, o sistema avalia sua elegibilidade para as subcategorias da Lei de Cotas, como as modalidades LB, destinadas a candidatos de baixa renda, e LI, independentes de renda. Por fim, quando aplicável, são consideradas as Ações Afirmativas Próprias ofertadas pela instituição.

Esse mecanismo produz o chamado remanejamento automático, ou transbordo, no qual candidatos cotistas com notas elevadas podem ocupar vagas de Ampla Concorrência, liberando vagas reservadas para os próximos candidatos elegíveis dentro de cada subcota. Quando há vagas remanescentes por ausência de candidatos aptos, sua redistribuição segue uma ordem hierárquica definida pelo Ministério da Educação. Essa ordem prioriza, inicialmente, as subdivisões internas da própria Lei de Cotas, como os grupos de pretos, pardos, indígenas, quilombolas e pessoas com deficiência. Esgotadas as possibilidades de alocação nas submodalidades reservadas, as vagas ociosas podem ser revertidas para a Ampla Concorrência. Na prática, a complexidade algorítmica e documental desse fluxo pode ocasionar ociosidade temporária de vagas, justificando o desenvolvimento de estratégias de alocação matemática que considerem explicitamente a estrutura de transbordo do sistema (BRASIL, 2023a). A ordem de prioridades adotada no SISU é apresentada na Tabela 1.

Tabela 1 – Hierarquia de remanejamento por modalidade de vaga

Modalidade da vaga	Ordem de remanejamento
LB_PPI	$LB_PP \succ LB_I \succ LB_Q \succ LB_PCD \succ LB_EP \succ LI_PPI \succ LI_PP \succ LI_I \succ LI_Q \succ LI_PCD \succ LI_EP \succ AC$
LB_PP	$LB_PPI \succ LB_I \succ LB_Q \succ LB_PCD \succ LB_EP \succ LI_PPI \succ LI_PP \succ LI_I \succ LI_Q \succ LI_PCD \succ LI_EP \succ AC$
LB_I	$LB_PPI \succ LB_PP \succ LB_Q \succ LB_PCD \succ LB_EP \succ LI_PPI \succ LI_PP \succ LI_I \succ LI_Q \succ LI_PCD \succ LI_EP \succ AC$
LB_Q	$LB_PPI \succ LB_PP \succ LB_I \succ LB_PCD \succ LB_EP \succ LI_PPI \succ LI_PP \succ LI_I \succ LI_Q \succ LI_PCD \succ LI_EP \succ AC$
LB_PCD	$LB_PPI \succ LB_PP \succ LB_I \succ LB_Q \succ LB_EP \succ LI_PPI \succ LI_PP \succ LI_I \succ LI_Q \succ LI_PCD \succ LI_EP \succ AC$
LB_EP	$LB_PPI \succ LB_PP \succ LB_I \succ LB_Q \succ LB_PCD \succ LI_PPI \succ LI_PP \succ LI_I \succ LI_Q \succ LI_PCD \succ LI_EP \succ AC$
LI_PPI	$LI_PP \succ LI_I \succ LI_Q \succ LI_PCD \succ LI_EP \succ LB_PPI \succ LB_PP \succ LB_I \succ LB_Q \succ LB_PCD \succ LB_EP \succ AC$
LI_PP	$LI_PPI \succ LI_I \succ LI_Q \succ LI_PCD \succ LI_EP \succ LB_PPI \succ LB_PP \succ LB_I \succ LB_Q \succ LB_PCD \succ LB_EP \succ AC$
LI_I	$LI_PPI \succ LI_PP \succ LI_Q \succ LI_PCD \succ LI_EP \succ LB_PPI \succ LB_PP \succ LB_I \succ LB_Q \succ LB_PCD \succ LB_EP \succ AC$
LI_Q	$LI_PPI \succ LI_PP \succ LI_I \succ LI_PCD \succ LI_EP \succ LB_PPI \succ LB_PP \succ LB_I \succ LB_Q \succ LB_PCD \succ LB_EP \succ AC$
LI_PCD	$LI_PPI \succ LI_PP \succ LI_I \succ LI_Q \succ LI_EP \succ LB_PPI \succ LB_PP \succ LB_I \succ LB_Q \succ LB_PCD \succ LB_EP \succ AC$
LI_EP	$LI_PPI \succ LI_PP \succ LI_I \succ LI_Q \succ LI_PCD \succ LB_PPI \succ LB_PP \succ LB_I \succ LB_Q \succ LB_PCD \succ LB_EP \succ AC$

Fonte: adaptado de BRASIL (2025a).

Embora essas políticas ampliem a inclusão, a fragmentação das vagas em diversas subcategorias pode dificultar o preenchimento total do sistema (Nogueira *et al.*, 2017).

2.1.6 Vagas Remanescentes e Maximização da Ocupação

Mesmo com algoritmos estáveis de emparelhamento, parte das vagas do SISU permanece ociosa ao final da alocação. Nos experimentos de Oliveira *et al.* (2026) com dados reais do SISU 2023 mais de 226 mil vagas e 1 milhão de candidatos métodos baseados no *Gale-Shapley* atingiram aproximadamente 98,2% de ocupação, deixando ainda milhares de vagas não preenchidas. Essa ociosidade decorre de fatores como a limitação de duas opções por candidato, a estrutura de empates entre modalidades e a distribuição geográfica desigual da demanda.

Nesse contexto, o problema *MAX-HRT* formaliza o objetivo de maximizar o número de candidatos alocados preservando as restrições de estabilidade e capacidade. A NP-dificuldade do problema, combinada com a escala nacional do SISU, torna inviável a aplicação direta de métodos exatos, motivando estratégias de decomposição e heurísticas incrementais detalhadas no Capítulo 4.

2.2 Problemas de *Matching* e Alocação Educacional

Esta seção introduz a base matemática utilizada para modelar a distribuição de candidatos em vagas universitárias.

2.2.1 *Stable Matching e o Algoritmo de Gale-Shapley*

O conceito de emparelhamento estável busca associar dois grupos de agentes, como alunos e faculdades ou homens e mulheres, de modo que a solução seja mutuamente aceitável. Dado um mercado com N agentes em cada grupo, cada um com uma lista de preferências estrita e completa sobre o grupo oposto, um emparelhamento M é dito estável quando não existem pares bloqueantes. Um par $(m_i, w_j) \notin M$ é bloqueante quando ambos preferem estar juntos a permanecer com seus parceiros atuais (Gale; Shapley, 1962).

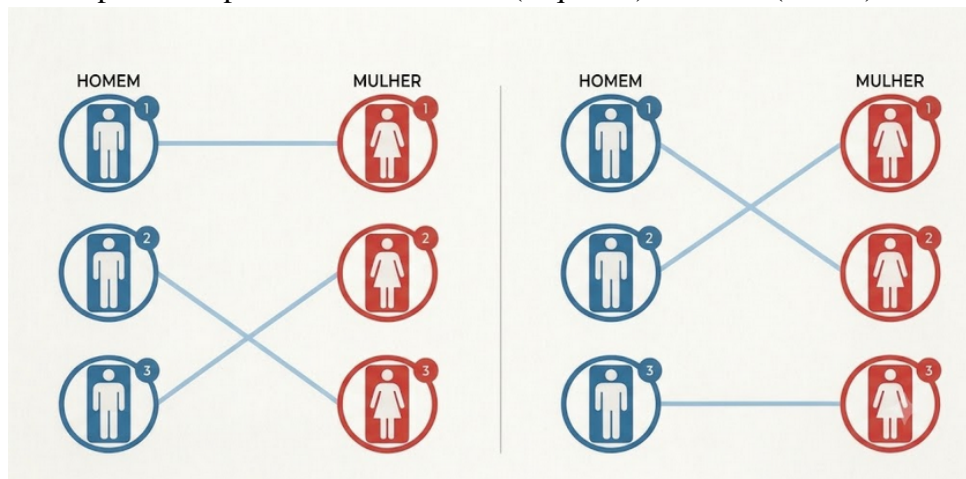
A Tabela 2 ilustra uma instância com $N = 3$, a partir da qual é possível comparar diferentes soluções. Na Figura 1, o emparelhamento à esquerda, $M = \{(m_1, w_3), (m_2, w_1), (m_3, w_2)\}$, não é estável, pois admite pares bloqueantes. Já o emparelhamento à direita, $M = \{(m_1, w_1), (m_2, w_3), (m_3, w_2)\}$, é estável, pois nenhum par externo possui incentivo bilateral de desvio.

Tabela 2 – Instância do Problema de Emparelhamento Estável (SMP)

Homem	Lista de preferências do homem	Mulher	Lista de preferências da mulher
m_1	(w_1, w_2, w_3)	w_1	(m_3, m_1, m_2)
m_2	(w_1, w_3, w_2)	w_2	(m_1, m_2, m_3)
m_3	(w_2, w_3, w_1)	w_3	(m_3, m_2, m_1)

Fonte: Fenoaltea *et al.* (2021).

Figura 1 – Exemplo de emparelhamento instável (esquerda) e estável (direita)



Fonte: Adaptado usando Gemini pro Fenoaltea *et al.* (2021).

O Algoritmo de Gale-Shapley resolve o problema de *Stable Matching* por meio do protocolo de aceitação diferida em cada rodada, cada proponente livre faz uma proposta à sua melhor opção ainda não consultada, e cada receptora retém provisoriamente o candidato mais

preferido, rejeitando os demais. O processo se repete até que não restem agentes livres, garantindo convergência para um emparelhamento estável ótimo para os proponentes. A formalização completa do mecanismo é apresentada a seguir.

2.2.2 Formalização do Algoritmo de Gale-Shapley

Para a formalização do mecanismo, considera-se um mercado composto por dois conjuntos finitos e disjuntos $H = \{h_1, h_2, \dots, h_n\}$, representando os proponentes, e $M = \{m_1, m_2, \dots, m_n\}$, representando as seletoras. O conjunto de preferências é definido como:

$$\mathcal{P} = \{\succ_h: h \in H\} \cup \{\succ_m: m \in M\},$$

onde cada $\succ_i$ é uma ordem linear estrita e completa sobre o conjunto oposto. O objetivo é encontrar um emparelhamento μ estável, isto é, sem pares bloqueantes um par $(h, m) \notin \mu$ é bloqueante se $h \succ_m \mu(m)$ e $m \succ_h \mu(h)$, ou seja, ambos preferem um ao outro em relação aos seus parceiros atuais (Gale; Shapley, 1962). O Algoritmo 1 formaliza esse mecanismo.

Algoritmo 1: Algoritmo de Gale-Shapley (orientado ao proponente)

Entrada: Conjuntos H e M ; listas de preferências $\mathcal{P}$

Saída: Emparelhamento estável μ

Inicialize todos os $h \in H$ e $m \in M$ como livres;

while existir algum $h \in H$ livre que ainda não propôs a todas as $m \in M$ **do**

$m \leftarrow$ mulher de maior preferência na lista de h ainda não consultada;

if m está livre **then**

(h, m) tornam-se engajados;

else

 // m está engajada com algum h' ; compara preferências

if $h \succ_m h'$ **then**

h' torna-se livre;

(h, m) tornam-se engajados;

else

h permanece livre;

 ;

 // m rejeita h

end

end

end

return o conjunto de pares engajados μ ;

2.2.3 Exemplo de execução Algoritmo de Gale-Shapley

Para ilustrar a execução, considera-se um mercado com $n = 5$ agentes em cada grupo;

$$H = \{h_1: \text{Tailan}, h_2: \text{Kaio}, h_3: \text{Rubens}, h_4: \text{João Vitor}, h_5: \text{Thalison}\}$$

$$M = \{m_1: \text{Fernanda}, m_2: \text{Cristina}, m_3: \text{Tatiely}, m_4: \text{Maria}, m_5: \text{Angélica}\}$$

O estado de engajamento ao final de cada iteração t é denotado por $E_t \subset H \times M$. As preferências dos agentes são as seguintes, destacando-se que h_5 e m_5 possuem escolha de topo mútua (Gubitoso; Lopes, 2017):

$$\left\{ \begin{array}{l} \mathcal{P}(h_1): m_1 \succ_{h_1} m_4 \succ_{h_1} m_3 \succ_{h_1} m_2 \succ_{h_1} m_5 \\ \mathcal{P}(h_2): m_2 \succ_{h_2} m_1 \succ_{h_2} m_4 \succ_{h_2} m_3 \succ_{h_2} m_5 \\ \mathcal{P}(h_3): m_1 \succ_{h_3} m_2 \succ_{h_3} m_3 \succ_{h_3} m_4 \succ_{h_3} m_5 \\ \mathcal{P}(h_4): m_2 \succ_{h_4} m_1 \succ_{h_4} m_3 \succ_{h_4} m_4 \succ_{h_4} m_5 \\ \mathcal{P}(h_5): m_5 \succ_{h_5} m_1 \succ_{h_5} m_2 \succ_{h_5} m_3 \succ_{h_5} m_4 \end{array} \right. \quad \left\{ \begin{array}{l} \mathcal{P}(m_1): h_1 \succ_{m_1} h_3 \succ_{m_1} h_4 \succ_{m_1} h_2 \succ_{m_1} h_5 \\ \mathcal{P}(m_2): h_2 \succ_{m_2} h_4 \succ_{m_2} h_3 \succ_{m_2} h_1 \succ_{m_2} h_5 \\ \mathcal{P}(m_3): h_3 \succ_{m_3} h_1 \succ_{m_3} h_2 \succ_{m_3} h_4 \succ_{m_3} h_5 \\ \mathcal{P}(m_4): h_4 \succ_{m_4} h_1 \succ_{m_4} h_2 \succ_{m_4} h_3 \succ_{m_4} h_5 \\ \mathcal{P}(m_5): h_5 \succ_{m_5} h_1 \succ_{m_5} h_2 \succ_{m_5} h_3 \succ_{m_5} h_4 \end{array} \right. \quad (2.3)$$

A execução segue o protocolo de aceitação diferida: em cada rodada, cada proponente livre propõe à sua melhor opção ainda não consultada, e cada receptora retém provisoriamente o candidato mais preferido, rejeitando os demais.

Iteração $t = 1$: h_1 e h_3 propõem a m_1 ; h_2 e h_4 propõem a m_2 ; h_5 propõe a m_5 .

- m_1 recebe $\{h_1, h_3\}$: como $h_1 \succ_{m_1} h_3$, retém h_1 e rejeita h_3 .
- m_2 recebe $\{h_2, h_4\}$: como $h_2 \succ_{m_2} h_4$, retém h_2 e rejeita h_4 .
- m_5 recebe apenas h_5 e o retém provisoriamente.

$$E_1 = \{(h_1, m_1), (h_2, m_2), (h_5, m_5)\}.$$

Iteração $t = 2$: Livres: $\mathcal{F} = \{h_3, h_4\}$. h_3 propõe a m_2 ; h_4 propõe a m_1 .

- m_1 : como $h_1 \succ_{m_1} h_4$, mantém h_1 e rejeita h_4 .
- m_2 : como $h_2 \succ_{m_2} h_3$, mantém h_2 e rejeita h_3 .

$$E_2 = E_1 = \{(h_1, m_1), (h_2, m_2), (h_5, m_5)\}.$$

Iteração $t = 3$: Livres: $\mathcal{F} = \{h_3, h_4\}$. Ambos propõem a m_3 .

- m_3 recebe $\{h_3, h_4\}$: como $h_3 \succ_{m_3} h_4$, retém h_3 e rejeita h_4 .

$$E_3 = \{(h_1, m_1), (h_2, m_2), (h_3, m_3), (h_5, m_5)\}.$$

Iteração $t = 4$: Livre: $\mathcal{F} = \{h_4\}$. h_4 propõe a m_4 , que está livre e o retém.

$$E_4 = \{(h_1, m_1), (h_2, m_2), (h_3, m_3), (h_4, m_4), (h_5, m_5)\}.$$

Como $\mathcal{F} = \emptyset$, o algoritmo encerra com o emparelhamento estável:

$$\mu^* = \{(h_1, m_1), (h_2, m_2), (h_3, m_3), (h_4, m_4), (h_5, m_5)\} \quad (2.4)$$

correspondendo aos pares (Tailan, Fernanda), (Kaio, Cristina), (Rubens, Tatiely), (João Vitor, Maria) e (Thalison, Angélica). A estabilidade decorre da inexistência de pares bloqueantes: para todo $(h_i, m_j) \notin \mu^*$, ou h_i prefere $\mu^*(h_i)$ a m_j , ou m_j prefere $\mu^*(m_j)$ a h_i . Em particular, (h_5, m_5) é uma escolha de topo mútua, com $\mu^*(h_5) = m_5$ e $\mu^*(m_5) = h_5$. Por ser orientado ao proponente, μ^* é ótimo para H : nenhum emparelhamento estável alternativo poderia oferecer a qualquer h_i uma parceira estritamente melhor (Gale; Shapley, 1962).

No contexto do SISU, candidatos assumem o papel de proponentes e cursos universitários o de receptores. As preferências dos candidatos são expressas por suas escolhas de curso e modalidade de concorrência, enquanto as instituições operam sob critérios estritos baseados no desempenho no ENEM. O mecanismo de aceitação diferida garante que a alocação final seja estável: não existe par (candidato, curso) em que ambos prefeririam estar juntos a permanecer com suas alocações atuais, assegurando a integridade do processo seletivo (Gale; Shapley, 1962).

2.3 O Problema *MAX-HRT*

Segundo Kwanashie e Manlove (2014), o problema *Hospitals/Residents Problem with Ties* (HRT) estende o modelo clássico de emparelhamento estável ao permitir que as listas de preferências dos agentes contenham empates, representando situações de indiferença entre alternativas. Uma instância de *HRT* é composta por um conjunto de residentes $R = \{r_1, r_2, \dots, r_{n_1}\}$ e um conjunto de hospitais $H = \{h_1, h_2, \dots, h_{n_2}\}$, onde cada hospital $h_j \in H$ possui uma capacidade $c_j \in \mathbb{Z}^+$ indicando o número máximo de residentes que pode receber. Um par (r_i, h_j) é dito aceitável se h_j figura na lista de preferências de r_i e vice-versa. Um emparelhamento M é um conjunto de pares aceitáveis tal que cada residente é associado a no máximo um hospital e nenhum hospital excede sua capacidade (Kwanashie; Manlove, 2014).

No modelo clássico *HR*, com listas de preferências estritas, todos os emparelhamentos estáveis possuem a mesma cardinalidade. Essa propriedade deixa de valer quando empates são introduzidos diferentes formas de resolver as indiferenças podem produzir emparelhamentos

estáveis de tamanhos distintos (Kwanashie; Manlove, 2014). O problema de encontrar um emparelhamento fracamente estável de cardinalidade máxima é denominado *MAX-HRT* e é classificado como NP-difícil (Kwanashie; Manlove, 2014).

No contexto desta pesquisa, o SISU é modelado como uma instância de grande escala do problema *MAX-HRT*. Os candidatos assumem o papel de residentes, enquanto os pares (curso, modalidade) de concorrência assumem o papel de hospitais, e as vagas ofertadas por modalidade de concorrência correspondem às capacidades c_j . As preferências dos candidatos são definidas por suas escolhas de curso, enquanto as instituições operam sob critério estrito de classificação pela nota do ENEM. Embora esse critério produza ordenações majoritariamente estritas, a existência de notas idênticas entre candidatos, combinada com múltiplas modalidades de concorrência e regras de remanejamento, introduz estruturas de indiferença que aproximam o problema à formulação *HRT*. Nesse cenário, o objetivo é encontrar uma alocação de cardinalidade máxima que seja fracamente estável, garantindo que nenhum candidato e curso preteridos se prefeririam mutuamente em relação às suas alocações vigentes.

2.3.1 Tipos de Estabilidade no Problema de Emparelhamento

Scott (2005) A estabilidade em problemas de alocação bilateral é caracterizada pela ausência de pares bloqueantes. Contudo, quando as listas de preferências contêm empates, a definição de bloqueio admite interpretações distintas, dando origem a três noções de estabilidade com diferentes graus de exigência (Scott, 2005).

- Estabilidade fraca: um par $(r_i, h_j) \notin M$ bloqueia M se cada membro do par prefere estritamente o outro ao seu parceiro atual. Um emparelhamento sem tal par bloqueante é dito fracamente estável. Toda instância HRT admite ao menos um emparelhamento fracamente estável, obtido pela quebra arbitrária dos empates e aplicação do algoritmo de Gale-Shapley (Scott, 2005; Kwanashie; Manlove, 2014).
- Super-estabilidade: um par $(r_i, h_j) \notin M$ bloqueia M se cada membro prefere o outro ao seu parceiro atual ou é indiferente entre eles. Um emparelhamento sem tal par é dito super-estável. É a condição mais restritiva e não existe em toda instância por exemplo, quando todas as listas de preferências formam um único empate completo, nenhum emparelhamento super-estável pode existir (Scott, 2005).
- Estabilidade forte: condição intermediária entre as duas anteriores. Um par $(r_i, h_j) \notin M$ bloqueia M se um dos membros prefere estritamente o outro ao seu parceiro atual enquanto

o outro é ao menos indiferente. Um emparelhamento sem tal par é dito fortemente estável. Todo emparelhamento super-estável é fortemente estável, e todo emparelhamento fortemente estável é fracamente estável; a recíproca, porém, não é necessariamente verdadeira (Scott, 2005).

Este trabalho adota a estabilidade fraca por ser a única noção que garante solução para toda instância HRT e sobre a qual o modelo de Programação Inteira de Kwanashie e Manlove (2014) é formulado. Dado que *MAX-HRT* é NP-difícil, métodos exatos convencionais tornam-se inviáveis na escala do SISU, motivando a estratégia de decomposição detalhada no Capítulo 4.

2.3.2 Formulação por Programação Inteira

Kwanashie e Manlove (2014) propõem uma formulação de Programação Inteira para o problema *MAX-HRT*, cujo objetivo é encontrar um emparelhamento fracamente estável de cardinalidade máxima, conforme ilustrado na Figura 2.

Figura 2: Modelo de Programação Inteira para o problema *MAX-HRT*

$$\begin{aligned}
 & \max \sum_{i=1}^{n_1} \sum_{h_j \in P(r_i)} x_{i,j} \\
 & \text{subject to} \\
 & 1. \sum_{h_j \in P(r_i)} x_{i,j} \leq 1 \quad (1 \leq i \leq n_1) \\
 & 2. \sum_{r_i \in P(h_j)} x_{i,j} \leq c_j \quad (1 \leq j \leq n_2) \\
 & 3. c_j \left(1 - \sum_{h_q \in S_{i,j}} x_{i,q} \right) - \sum_{r_p \in T_{i,j}} x_{p,j} \leq 0 \quad (1 \leq i \leq n_1, h_j \in P(r_i)) \\
 & x_{i,j} \in \{0, 1\}
 \end{aligned}$$

Fonte: Kwanashie e Manlove (2014).

A formulação utiliza a seguinte notação:

- $R = \{r_1, \dots, r_{n_1}\}$: conjunto de residentes (candidatos);
- $H = \{h_1, \dots, h_{n_2}\}$: conjunto de hospitais (cursos/modalidades);
- $P(r_i)$: hospitais aceitáveis para r_i ; $P(h_j)$: residentes aceitáveis para h_j ;
- c_j : capacidade de vagas do hospital h_j ;
- $x_{i,j} \in \{0, 1\}$: variável binária que indica se r_i é alocado em h_j ;

- $S_{i,j}$: hospitais que r_i prefere tanto quanto ou mais que h_j ;
- $T_{i,j}$: residentes que h_j prefere tanto quanto ou mais que r_i .

O objetivo é maximizar o número total de residentes alocados:

$$\max \sum_{i=1}^{n_1} \sum_{h_j \in P(r_i)} x_{i,j}$$

sujeito a:

1. Alocação única: cada residente é alocado em no máximo um hospital:

$$\sum_{h_j \in P(r_i)} x_{i,j} \leq 1, \quad 1 \leq i \leq n_1$$

2. Capacidade: o número de residentes alocados não excede c_j :

$$\sum_{r_i \in P(h_j)} x_{i,j} \leq c_j, \quad 1 \leq j \leq n_2$$

3. Estabilidade: impede a existência de pares bloqueantes:

$$c_j \left(1 - \sum_{h_q \in S_{i,j}} x_{i,q} \right) - \sum_{r_p \in T_{i,j}} x_{p,j} \leq 0, \quad 1 \leq i \leq n_1, h_j \in P(r_i)$$

A restrição de estabilidade opera da seguinte forma se r_i não está alocado em nenhum hospital de $S_{i,j}$ ou seja, não obteve nenhuma de suas opções preferidas em relação a h_j então h_j deve estar completamente preenchido por residentes de $T_{i,j}$, isto é, candidatos que ele considera ao menos tão bons quanto r_i . Caso contrário, o par (r_i, h_j) constituiria um par bloqueante, violando a estabilidade fraca (Kwanashie; Manlove, 2014)

Embora a formulação garanta soluções ótimas e estáveis, o custo computacional cresce significativamente em instâncias de grande escala. Como observado por Kwanashie e Manlove (2014), a resolução exata torna-se inviável para cenários nacionais como o SISU, motivando a estratégias de decomposição via *K-Means Constrained* ou grafos detalhadas no Capítulo 4.3.

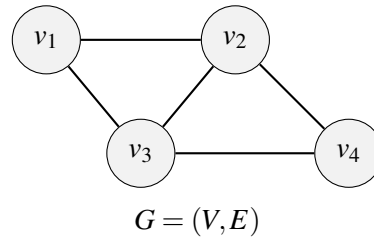
2.4 Teoria de Grafos e Clusterização de Cursos

2.4.1 Conceitos Básicos de Grafos

A teoria de grafos fornece uma linguagem formal para modelar estruturas de relacionamento entre entidades, sendo amplamente utilizada para representar relações par a par em

diferentes contextos. Um grafo $G = (V, E)$ é um par composto por um conjunto de vértices V e um conjunto de arestas E , em que cada aresta liga dois vértices de V (Gladkov, 2018). Em grafos simples, não há laços nem múltiplas arestas entre o mesmo par de vértices (Bondy; Murty, 1976). Quando as arestas possuem direção, obtém-se um dígrafo, quando não possuem, representam apenas adjacência entre vértices (Wilson, 1979).

Figura 3: Exemplo simples de grafo não direcionado.



Fonte: Elaborado pelo autor.

Neste trabalho, o SISU é modelado por grafos em dois níveis distintos. No nível da teoria de emparelhamento, a relação candidatos–vagas é representada como um grafo bipartido, em que o conjunto de vértices é particionado em dois subconjuntos disjuntos e cada aresta liga um vértice de um lado a um vértice do outro (Gladkov, 2018). Essa estrutura corresponde ao problema HRT, no qual um lado representa candidatos e o outro representa pares curso–modalidade de concorrência.

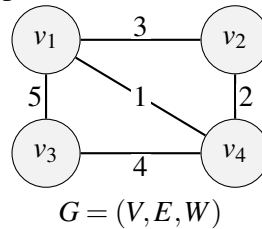
Para a etapa de decomposição proposta neste trabalho, considera-se um grafo de co-inscrição entre cursos, em que cada vértice representa um curso do SISU e existe uma aresta entre dois cursos sempre que algum candidato os escolhe simultaneamente em suas opções de inscrição. Esse grafo é naturalmente ponderado, o peso da aresta entre dois cursos indica o número de candidatos inscritos em ambos simultaneamente, sintetizando a intensidade de co-inscrição entre eles.

A noção de componente conexa constitui um primeiro conceito de decomposição de grafos uma componente conexa é um subgrafo maximal em que qualquer par de vértices está ligado por algum caminho (West *et al.*, 2001). Em grafos muito densos, porém, é comum que a maior parte dos vértices pertença a uma única grande componente, o que limita a utilidade dessa decomposição para fins de viabilidade computacional. Para obter subproblemas menores e balanceados, torna-se necessário recorrer a técnicas mais elaboradas de particionamento de grafos, discutidas na subseção seguinte.

2.4.2 Particionamento e Cortes em Grafos

Problemas de particionamento de grafos buscam dividir o conjunto de vértices em partes, minimizando o peso total das arestas cortadas, possivelmente sob restrições de balanceamento entre os tamanhos das partes. Em um grafo ponderado $G = (V, E, W)$, um corte é uma bipartição do conjunto de vértices V em dois subconjuntos disjuntos S e $V \setminus S$ o peso do corte é a soma dos pesos das arestas que cruzam entre os dois lados (Zhu; Segarra, 2022). O problema clássico de *minimum cut* procura o corte de menor peso e está na base de diversas aplicações em redes, segmentação de imagens e análise de confiabilidade (Zhu; Segarra, 2022).

Figura 4: Exemplo simples de grafo ponderado.



Fonte: Elaborado pelo autor.

No contexto deste trabalho, o particionamento do grafo de co-inscrição em K partes corresponde diretamente à decomposição da instância do SISU em subinstâncias independentes. As arestas cortadas pelo particionamento representam candidatos cujas duas opções de curso ficam em clusters distintos os denominados *candidatos de fronteira*. Minimizar o peso total do corte é, portanto, equivalente a minimizar o número de candidatos de fronteira, reduzindo as dependências entre subinstâncias e simplificando o processo incremental de migração descrito na Seção 4.5.

Variações mais ricas consideram particionar o grafo em k partes com restrições de balanceamento e minimização do custo de corte. No problema de *k-balanced graph partitioning*, busca-se particionar os vértices em k subconjuntos aproximadamente iguais minimizando o número de arestas cortadas (Feldmann; Foschini, 2012). Resultados de complexidade mostram que esse problema é NP-difícil mesmo em classes restritas de grafos, como árvores, o que motiva o uso de heurísticas e métodos aproximativos em aplicações de grande escala (Feldmann; Foschini, 2012). No presente trabalho, essa restrição de balanceamento é essencial *clusters* de tamanho equilibrado garantem que o modelo de Programação Inteira possa ser aplicado em cada subinstância dentro de um tempo computacional viável.

2.4.3 *Clusterização e Métodos Baseados em Min-cut*

De modo mais amplo, o particionamento de grafos está associado a tarefas de clusterização de dados. Em muitos cenários, as similaridades par a par entre objetos são representadas por um grafo ponderado, e clusterizar os dados equivale a encontrar uma partição que agrupe vértices fortemente conectados e separe vértices fracamente conectados. Nesse contexto, o problema de particionamento pode ser formulado de modo a maximizar a coerência interna dos clusters ou minimizar o custo de corte entre grupos (Ding *et al.*, 2001). Por exemplo, funções objetivo baseadas em *min-max cut* levam a partições relativamente balanceadas e têm sido propostas como alternativa a métodos puramente espectrais tradicionais (Ding *et al.*, 2001).

2.4.4 *Modelagem do SISU como Grafo Curso–Curso*

A decomposição proposta para o presente trabalho pode ser interpretada justamente como uma forma de particionamento do grafo curso–curso induzido pelas co-inscrições. Ao representar cada curso como um vértice e conectar dois cursos por uma aresta sempre que existem candidatos que escolheram ambos, constrói-se um grafo ponderado em que o peso da aresta reflete a intensidade dessa co-inscrição. Clusterizar os cursos equivale então a encontrar uma partição dos vértices em subconjuntos de tamanho limitado (os *clusters*), de forma análoga a problemas de *k*-particionamento balanceado, mas com uma função objetivo específica minimizar o número de candidatos de fronteira, isto é, minimizar o número de arestas que ligam cursos pertencentes a *clusters* distintos.

2.4.5 *Candidatos Internos, Candidatos de Fronteira e Impacto no MAX-HRT*

Do ponto de vista da teoria de grafos, candidatos internos são aqueles cujas opções de curso pertencem ao mesmo *cluster*, correspondendo a arestas inteiramente contidas em um subgrafo induzido. Já candidatos de fronteira são aqueles cujas opções pertencem a *clusters* distintos, induzindo arestas de corte entre diferentes grupos. Assim, reduzir o número de candidatos de fronteira é análogo a buscar partições com baixo custo de corte, conceito central em problemas de corte mínimo e particionamento equilibrado (Zhu; Segarra, 2022; Feldmann; Foschini, 2012). Ao mesmo tempo, a restrição de tamanho máximo dos *clusters* desempenha papel semelhante às restrições de cardinalidade presentes em problemas de corte mínimo com cardinalidade limitada (Andreotti *et al.*, 2019).

Essa visão em termos de grafos esclarece o papel da etapa de clusterização na escalabilidade do modelo *MAX-HRT*. Ao decompor o grafo curso–curso em *clusters* relativamente pequenos, obtém-se uma coleção de subgrafos sobre os quais é possível resolver de forma exata o modelo de Programação Inteira para *MAX-HRT*, o que seria inviável na instância nacional completa. O impacto da escolha da partição é duplo: por um lado, *clusters* menores reduzem o custo computacional por subproblema; por outro, cortes com muitos candidatos de fronteira aumentam a necessidade de migração entre *clusters* e podem comprometer a qualidade global da solução. Em termos de teoria de grafos, essa tensão é análoga ao compromisso clássico entre qualidade de corte e balanceamento das partes em problemas de particionamento (Ding *et al.*, 2001; Feldmann; Foschini, 2012).

Em síntese, a fundamentação em teoria de grafos permite interpretar a estratégia de decomposição deste trabalho como uma forma específica de particionamento de um grafo ponderado de co-inscrição. Nessa representação, os conceitos de vértices, arestas, pesos, cortes e *clusters* correspondem, respectivamente, a cursos, co-inscrição de candidatos, intensidade de compartilhamento de candidatos, candidatos de fronteira e subproblemas locais de *MAX-HRT*.

2.5 Algoritmo Proposto de Decomposição e Mecanismo de Migração

Para viabilizar a aplicação do modelo de Programação Inteira do *MAX-HRT* na escala do SISU, a instância global é decomposta em subinstâncias menores por meio do agrupamento de cursos. Cada subinstância é resolvida inicialmente de forma exata pelo modelo de programação inteira. Em seguida, candidatos rejeitados em sua primeira opção e inscritos em cursos de *clusters* distintos, denominados candidatos de fronteira, migram para o *cluster* de sua segunda opção. Nesse *cluster*, são executadas iterações incrementais baseadas no algoritmo de *Gale-Shapley*, até que não ocorram novas migrações.

2.5.1 Clusterização de Cursos via *K-Means Constrained*

A clusterização é utilizada como mecanismo de decomposição da instância global, permitindo dividir o problema original em subinstâncias menores e mais tratáveis. Algoritmos tradicionais de agrupamento, entretanto, não garantem equilíbrio entre os grupos gerados, podendo produzir *clusters* excessivamente grandes e inviabilizar a resolução exata das subinstâncias. Para contornar essa limitação, utiliza-se o *K-Means Constrained*, que incorpora restrições de

cardinalidade durante o processo de agrupamento e favorece a formação de *clusters* de tamanho equilibrado (Levy-Kramer, 2018).

A similaridade entre cursos é definida a partir do compartilhamento de candidatos inscritos simultaneamente em ambos. Seja R_c o conjunto de candidatos associados ao curso c , a distância entre dois cursos c e c' é definida como:

$$d(c, c') = \frac{1}{1 + |R_c \cap R_{c'}|} \quad (2.5)$$

Essa formulação aproxima cursos com elevada interseção de candidatos, favorecendo que candidatos com preferências correlacionadas permaneçam no mesmo cluster. Cada curso é representado por um vetor de co-inscrições normalizado $\mathbf{v}_c \in \mathbb{R}^{|C|}$. A construção da matriz de similaridade exige operações de elevada dimensionalidade; conforme Cano (2018), o paralelismo das unidades de processamento gráfico (GPU) supera as limitações de abordagens em CPU para esse tipo de operação (Cano, 2018).

2.5.2 Algoritmo de Decomposição e Migração de Candidatos de Fronteira

Com os clusters definidos, a instância global é processada em duas fases. Na primeira, o modelo de programação inteira é executado de forma exata em cada cluster. Na segunda, candidatos de fronteira isto é, candidatos com duas opções de curso em clusters distintos que foram rejeitados na primeira opção migram para o cluster correspondente à segunda opção, onde são executadas iterações incrementais do Gale-Shapley. O algoritmo termina em número finito de iterações, pois cada candidato troca de cluster no máximo uma vez. A estabilidade fraca da solução final é avaliada experimentalmente e depende da ausência de pares bloqueantes após o processo de migração. O Algoritmo 2 formaliza esse processo.

Fonte: Fonte: elaborado pelo autor, com base na formulação *MAX-HRT* de (Kwanashie; Manlove, 2014)

2.5.3 Algoritmos Incrementais de Matching

Nas iterações subsequentes à primeira, utilizam-se algoritmos incrementais de emparelhamento baseados no Gale-Shapley para processar os candidatos de fronteira migrados. Esses métodos partem da solução existente no cluster e processam apenas os novos candidatos e aqueles eventualmente desalocados durante a execução, evitando recomputações completas

Algoritmo 2: Decomposição MAX-HRT com migração de candidatos de fronteira

Entrada: Candidatos R ; cursos C ; hospitais H ; capacidades q_h ; preferências $\mathcal{P}_R$ e $\mathcal{P}_H$; clusters $K_1, \dots, K_p$
Saída: Emparelhamento μ fracamente estável

```
// Inicialização
foreach candidato  $r \in R$  do
     $i \leftarrow \text{cluster}(c_1(r))$ ;
    if  $r$  é de fronteira then
         $\text{ativo}(r) \leftarrow c_1(r)$ ;
         $\text{estado}(r) \leftarrow \text{PRIMEIRA\_OPÇÃO}$ ;
    else
         $\text{ativo}(r) \leftarrow \text{todos os cursos de } r \text{ em } K_i$ ;
         $\text{estado}(r) \leftarrow \text{INTERNO}$ ;
    end
    Adicione  $r$  a  $R_i$ ;
end

// Primeira iteração: solução exata por IP
foreach cluster  $K_i$  do
    Construa a subinstância  $S_i$  com  $R_i$ ,  $H_i$  e cursos ativos;
     $M_i \leftarrow \text{IP}(S_i)$ ;
end
 $\mu \leftarrow \bigcup_i M_i$ ;
 $Q \leftarrow \{r \in R : r \text{ é de fronteira, } \text{estado}(r) = \text{PRIMEIRA\_OPÇÃO}, r \notin \mu\}$ ;

// Iterações incrementais: migração de candidatos de fronteira
while  $Q \neq \emptyset$  do
    Dirty  $\leftarrow \emptyset$ ;
    foreach  $r \in Q$  do
        Remova  $r$  de  $R_{\text{cluster}(c_1(r))}$ ;
        Adicione  $r$  a  $R_{\text{cluster}(c_2(r))}$ ;
         $\text{ativo}(r) \leftarrow c_2(r)$ ;
         $\text{estado}(r) \leftarrow \text{SEGUNDA\_OPÇÃO}$ ;
        Adicione  $\text{cluster}(c_2(r))$  a Dirty;
    end
    foreach cluster  $K_i \in \text{Dirty}$  do
         $M_i \leftarrow \text{GaleShapleyIncremental}(K_i, M_i, \text{Novos}_i)$ ;
    end
     $\mu \leftarrow \bigcup_i M_i$ ;
     $Q \leftarrow \{r \in R : r \text{ é de fronteira, } \text{estado}(r) = \text{PRIMEIRA\_OPÇÃO}, r \notin \mu\}$ ;
end
return  $\mu$ ;
```

(Oliveira *et al.*, 2026). Os critérios avaliados diferem na forma como resolvem os empates nas listas de preferência dos candidatos.

2.5.4 ASISU

O critério ASISU¹ serve como *baseline*. Os empates entre modalidades de um mesmo curso são resolvidos pela hierarquia fixa definida no edital do SISU, produzindo listas estritas sobre as quais o Gale-Shapley incremental é executado (Oliveira *et al.*, 2026).

Algoritmo 3: LCF Incremental (Least Competitive First)

Entrada: Candidatos R , Hospitais H , Capacidades $\{c_j\}$, Solução atual M_i

Saída: Emparelhamento atualizado μ

// Pré-cálculo da razão de competitividade

foreach hospital $h_j \in H$ **do**

$Razao(h_j) \leftarrow |P(h_j)| \div c_j$;

end

foreach candidato $r \in R$ **do**

 Ordene as modalidades empataadas de cada curso em ordem crescente de $Razao(h_j)$;

end

Execute o Gale-Shapley incremental partindo de M_i ;

return μ ;

2.5.5 LCF (*Least Competitive First*)

Fonte: adaptado de Oliveira *et al.* (2026).

O LCF² resolve os empates priorizando as modalidades com menor razão entre candidatos elegíveis e vagas disponíveis. A intuição é que modalidades menos concorridas têm maior risco de ociosidade; ao processá-las primeiro, busca-se maximizar o preenchimento total das vagas. Este critério é adotado neste trabalho por apresentar o melhor desempenho entre as heurísticas determinísticas avaliadas em Oliveira *et al.* (2026), alocando cinco candidatos a mais que a *baseline* nos experimentos com dados reais do SISU 2023.

2.5.6 ODF (*Observed Deficit First*)

Fonte: adaptado de Oliveira *et al.* (2026).

O ODF³ é uma heurística de duas etapas: primeiro executa o ASISU para estimar quais hospitais apresentam maior déficit de ocupação; em seguida, reordena as preferências dos candidatos para priorizar esses hospitais em uma segunda execução completa do Gale-Shapley. O ODF incorpora informação endógena da própria alocação, capturando parcialmente efeitos globais da interação entre cursos, modalidades e candidatos (Oliveira *et al.*, 2026).

¹ Código-fonte disponível em: <https://github.com/Naliat/TCC1/blob/main/heuristicas/asisu.py>.

² Código-fonte disponível em: <https://github.com/Naliat/TCC1/blob/main/heuristicas/lcf.py>.

³ Implementação disponível em: <https://github.com/Naliat/TCC1/blob/main/heuristicas/odf.py>.

Algoritmo 4: ODF Incremental (Observed Deficit First)

Entrada: Candidatos R , Hospitais H , Capacidades $\{c_j\}$, Solução atual M_i

Saída: Emparelhamento atualizado μ

// Fase de diagnóstico: usa solução ASISU como referência

Seja M_{ref} a solução obtida pelo critério ASISU;

foreach hospital $h_j \in H$ **do**

$\delta(h_j) \leftarrow c_j - |M_{\text{ref}}(h_j)|$;

end

// Reordenação das preferências

foreach candidato $r \in R$ **do**

 Ordene as modalidades empatadas em ordem decrescente de $\delta(h_j)$;

 Em caso de empate em δ , use a hierarquia ASISU como desempate;

end

Execute o Gale-Shapley incremental partindo de M_i ;

return μ ;

2.5.7 Exemplo Ilustrativo de Formação de Clusters

Para ilustrar o funcionamento do agrupamento, considere três candidatos e três cursos (Figura 5):

- Candidato 1: primeira opção Medicina, segunda opção Enfermagem;
- Candidato 2: primeira opção Medicina, segunda opção Enfermagem;
- Candidato 3: primeira opção Medicina, segunda opção Ciência da Computação.

A matriz de co-inscrição é construída conforme a Tabela 3, onde cada entrada indica o número de candidatos inscritos simultaneamente nos dois cursos correspondentes.

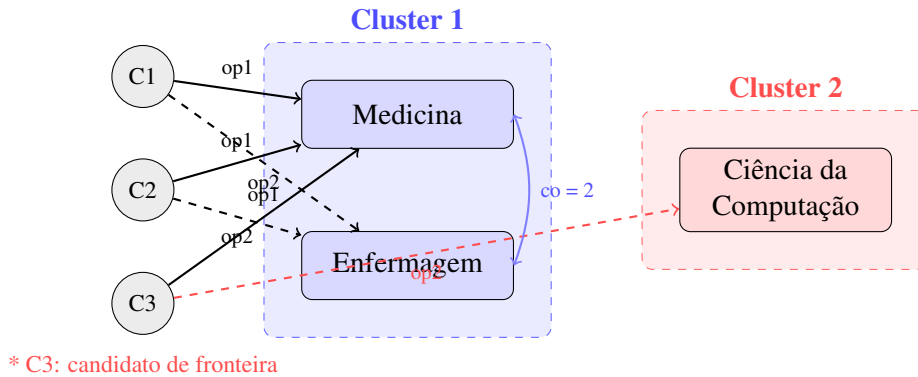
Tabela 3: Matriz de co-inscrição entre os cursos do exemplo.

	Medicina	Enfermagem	Comp.
Medicina	—	2	1
Enfermagem	2	—	0
Ciência da Computação	1	0	—

Fonte: Elaborado pelo autor

Medicina e Enfermagem possuem co-inscrição igual a 2, resultando em distância $d = 1/(1+2) \approx 0,33$, e são alocadas no mesmo cluster. Ciência da Computação possui co-inscrição baixa com ambos os cursos, resultando em distância maior e sendo alocada em cluster separado. O Candidato 3, com opções em clusters distintos, é classificado como *candidato de fronteira*.

Figura 5: Exemplo de formação de clusters. Candidatos 1 e 2 geram alta co-inscrição, mantendo Medicina e Enfermagem no mesmo cluster. C3 é candidato de fronteira por ter segunda opção em cluster distinto.



Fonte: Elaborado pelo autor

2.5.8 Mecanismo de Migração e Convergência

A presença de candidatos de fronteira introduz dependências entre *clusters*, uma vez que um mesmo candidato pode possuir opções de cursos em grupos distintos. Para tratar esse problema, o algoritmo adota um mecanismo de migração incremental: candidatos rejeitados por todos os hospitais do curso de primeira opção são transferidos para o *cluster* associado à segunda opção, onde participam de uma nova execução local do Gale-Shapley com critério de desempate LCF, conforme detalhado na Seção 2.5.5.

Formalmente, define-se a fila de migração Q como o conjunto de candidatos de fronteira rejeitados em todos os hospitais do *cluster* de origem. A cada iteração, os candidatos em Q migram para o *cluster* alternativo. Um *cluster* que recebe novos candidatos é denominado *dirty* e é reprocessado incrementalmente, partindo da solução existente e processando apenas os candidatos recém-chegados e os eventualmente desalocados durante a execução.

O processo termina quando $Q = \emptyset$. A convergência é garantida em número finito de iterações: como cada candidato possui no máximo duas opções de curso, cada um troca de *cluster* no máximo uma vez, impossibilitando ciclos de realocação. Quanto à estabilidade, o procedimento busca preservar a estabilidade fraca ao permitir a migração apenas após a rejeição do candidato em sua primeira opção e ao reprocessar localmente os *clusters* afetados por meio do *Gale-Shapley* incremental. Ainda assim, a estabilidade global da solução deve ser verificada ao final do processo, especialmente em relação a possíveis pares bloqueantes envolvendo candidatos e vagas de *clusters* distintos (Oliveira *et al.*, 2026).

3 TRABALHOS RELACIONADOS

Ao analisar a literatura atual foram identificados temas relevantes sobre algoritmos e heurísticas para maximizar o vies de alocação de vagas. Nesta seção, são apresentados esses estudos, mostrando suas contribuições e a forma como se relacionam com o presente trabalho.

3.1 *College admissions and the stability of marriage*

Gale e Shapley (1962), realizaram uma avaliação pioneira sobre a estabilidade em sistemas de admissão e casamentos, introduzindo o conceito de que um emparelhamento é estável apenas quando não existem pares que prefiram um ao outro em detrimento de suas alocações atuais. O foco central foi demonstrar matematicamente que, em qualquer mercado bilateral, existe ao menos uma solução estável que satisfaz as restrições de preferência dos agentes envolvidos.

A metodologia desta pesquisa consistiu na formulação de um algoritmo iterativo, hoje conhecido como Algoritmo de Aceitação Diferida. Logo, através de uma abordagem puramente lógica e dedutiva, os autores simularam o processo de propostas e rejeições temporárias, garantindo que o resultado final fosse ótimo para um dos lados do mercado (proponentes) e mantivesse a integridade do sistema contra desvios individuais (Gale; Shapley, 1962).

Os resultados deste estudo indicaram que a estabilidade é alcançável mesmo em larga escala e que o desenho do mecanismo dita quem será favorecido no processo. O estudo de (Gale; Shapley, 1962) relaciona-se diretamente com o presente trabalho por fornecer o algorítmico básico chamado de Gale-Shapley.

3.2 *The Hospitals/Residents Problem with Ties*

Irving *et al.* (2000), realizaram uma contribuição fundamental ao expandir o modelo de Gale-Shapley para cenários onde existem preferências com indiferenças, ou seja, empates. Os autores identificaram que o sistema original falha ao encontrar uma solução estável quando os agentes possuem notas ou preferências idênticas, permitindo que a alocação seja ambígua. O foco central foi estabelecer critérios de estabilidade que funcionem quando a ordenação estrita não é possível, garantindo que o sistema ainda possa oferecer uma solução tecnicamente válida.

A metodologia desta pesquisa consistiu na formulação de novas definições de estabilidade, como a "estabilidade fraca", que relaxa as condições do problema sem perder a

essência da alocação. Através de algoritmos de busca e análise de estruturas de grafos, os autores demonstraram como lidar com os empates, permitindo que o processo de decisão seja processado mesmo diante de instâncias complexas onde a igualdade de notas é frequente.

Os resultados deste estudo indicaram que a introdução de critérios de desempate e novas definições de estabilidade torna o modelo muito mais próximo da realidade de processos seletivos em larga escala como o SISU. O estudo de (Irving *et al.*, 2000) relaciona-se diretamente com o presente trabalho por fornecer o arcabouço necessário para que o algoritmo de alocação do SISU trate, de forma justa e determinística, a convergência de candidatos.

3.3 *An Integer Programming Approach to the Hospitals/Residents Problem with Ties*

Kwanashie e Manlove (2014), realizaram um avanço significativo ao converter o problema de *Hospitals/Residents with Ties* em um modelo de otimização matemática robusto. Os autores identificaram que, embora algoritmos baseados em *matching* sejam eficientes, eles frequentemente falham em encontrar a alocação que preenche o maior número possível de vagas. O foco central foi transpor as limitações dos algoritmos clássicos para o campo da otimização exata, buscando maximizar a ocupação total do sistema sob restrições de estabilidade.

A metodologia desta pesquisa consistiu na formulação de um modelo de Programação Inteira, capaz de resolver o problema de cardinalidade máxima MAX-HRT em instâncias com empates. Através da definição de variáveis binárias e restrições lineares que modelam a aceitabilidade e a estabilidade, os autores desenvolveram uma abordagem que permite ao solver encontrar a solução ótima, superando as heurísticas que frequentemente ficam presas em ótimos locais em cenários de alta competitividade (Kwanashie; Manlove, 2014).

Os resultados deste estudo indicaram que o uso de Programação Inteira permite preencher significativamente mais vagas do que os algoritmos tradicionais, garantindo simultaneamente o respeito às preferências dos agentes. O estudo de Kwanashie e Manlove (2014) relaciona-se diretamente com o presente trabalho por servir como a base algorítmica exata para a otimização da alocação de vagas do SISU, permitindo que a solução proposta atinja a cardinalidade máxima e resolva conflitos de vagas ociosas com precisão matemática.

3.4 *SISU Allocation with Indifference among Admission Modalities: a MAX-HRT Model and Comparison of Tie-Breaking Criteria*

Segundo Oliveira *et al.* (2026) foi investigado a aplicação de modelos de emparelhamento estável no contexto das novas regras de transbordo do SISU, BRASIL (2023b). O estudo focou em resolver o problema de indiferença entre modalidades de admissão, propondo um modelo denominado *MAX-HRT* para gerenciar o fluxo dinâmico de candidatos entre as camadas de cotas e ampla concorrência.

A metodologia empregada envolveu a simulação do algoritmo de Gale e Shapley (1962) adaptado para lidar com múltiplos critérios de desempate e a hierarquia de prioridades estabelecida pela legislação brasileira. Os autores compararam diferentes estratégias de ordenação, demonstrando como a configuração técnica do algoritmo impacta diretamente a estabilidade das vagas e a justiça distributiva no processo de seleção.

Os resultados obtidos revelaram que, embora o modelo atinja uma taxa de ocupação superior a 98%, o sistema ainda apresenta lacunas estruturais, resultando em 3.963 vagas ociosas e 872 combinações de curso/modalidade sem nenhum candidato alocado. Este trabalho é a base direta da presente pesquisa, pois fornece o *baseline* estatístico e identifica os pontos de falha no preenchimento de vagas (Oliveira *et al.*, 2026).

3.5 *Multi-Level Partitioning and Distribution of the Assignment Problem for Large-Scale Multi-Robot Task Allocation*

Liu e Shell (2012) Esse trabalho apresenta contribuições significativas para a resolução do problema de alocação de tarefas em sistemas de múltiplos robôs em larga escala, propondo um algoritmo que lida eficientemente com falhas e cenários dinâmicos. A principal inovação do estudo é a identificação de uma nova formulação de particionamento baseada em hipergrafos e matrizes, que permite uma alocação de tarefas multinível e top-down. Com isso, os autores conseguiram criar uma solução naturalmente distribuída que combina os méritos de abordagens centralizadas e descentralizadas, capaz de resolver alocações dinâmicas propagando mudanças na matriz de utilidade apenas entre partições inter-relacionadas e explorando propriedades de esparsidade para reduzir drasticamente a comunicação e a computação globais

A metodologia desta pesquisa baseia-se em uma estratégia de "dividir para conquistar" implementada em duas etapas principais. Na primeira etapa, realiza-se uma filtragem

que descarta utilidades irrelevantes para gerar uma matriz esparsa, seguida por uma fase de agrupamento (utilizando o algoritmo hMeTiS) que particiona o problema global em K subproblemas, agrupando pares de robô-tarefa fortemente conectados em blocos independentes. Na segunda etapa, a alocação de tarefas é paralelizada e refinada localmente dentro de cada cluster particionado, aplicando algoritmos exatos tradicionais (como o Algoritmo Húngaro). Para lidar com ambientes dinâmicos de forma contínua, o algoritmo monitora a mudança na densidade das utilidades e aciona o reparticionamento apenas entre os blocos que se inter-relacionaram devido à movimentação dos alvos ou robôs, evitando recálculos da rede inteira

Os resultados deste estudo mostraram que dividir o problema em partes menores traz um enorme ganho de velocidade e eficiência, quase sem perder a qualidade da alocação final. As simulações provaram que manter apenas uma pequena parcela das informações mais importantes (como focar apenas nas tarefas mais próximas aos robôs e descartar as distantes) já é o bastante para garantir decisões altamente precisas. Em termos de desempenho prático, o método proposto reduziu drasticamente o tempo de execução e a necessidade de comunicação entre a equipe, superando os algoritmos tradicionais que tentam resolver tudo de uma só vez. Além disso, em cenários onde as coisas mudam o tempo todo, os testes revelaram que reorganizar apenas os pequenos grupos que sofreram alterações entregou resultados excelentes. Essa estratégia de ajuste local gerou soluções quase tão boas quanto fazer o recálculo do sistema inteiro, mas economizando uma quantidade imensa de esforço computacional.

3.6 Análise comparativa

Quadro 01: Comparativo de contribuições acadêmicas.

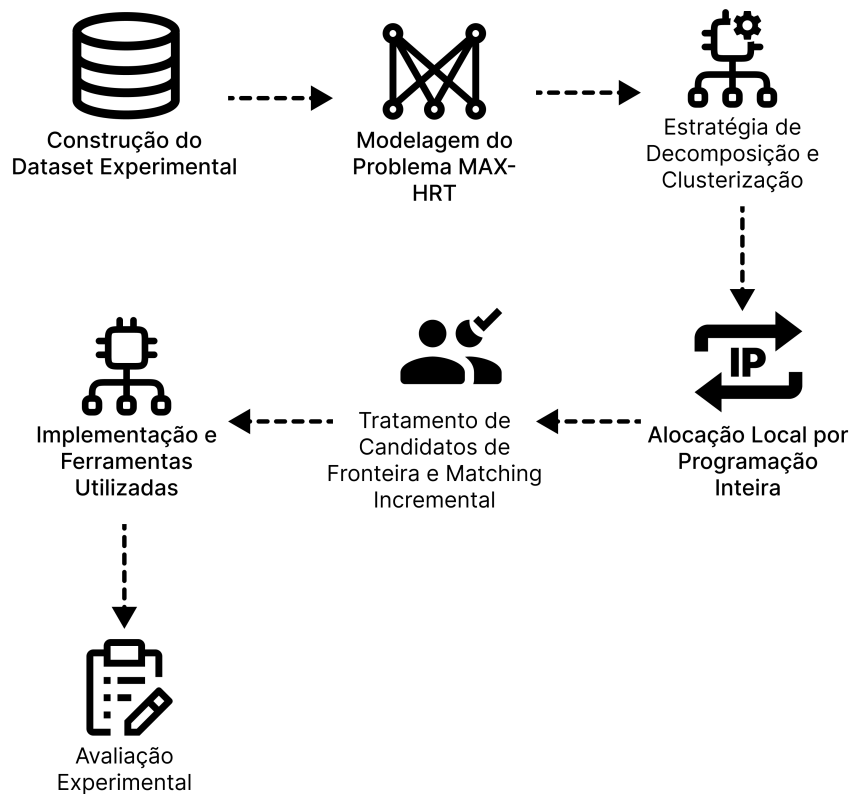
Autor	Fundamentação de Matching	Tratamento de Empates	Otimização (MAX-HRT)	Estratégia de Particionamento	Aplicação no SISU
(Gale; Shapley, 1962)	Propõe o algoritmo de aceitação diferida para emparelhamento estável.	Não consta.	Não consta.	Não consta.	Não consta.
(Irving <i>et al.</i> , 2000)	Estende a modelagem HR para considerar hospitais com múltiplas vagas.	Define formalmente os conceitos de estabilidade fraca, forte e super-estabilidade.	Não consta.	Não consta.	Não consta.
(Kwanashie; Manlove, 2014)	Apresenta a formulação do problema MAX-HRT.	Trata listas de preferências com empates matematicamente.	Apresenta modelo de Programação Inteira exato para alocação.	Não consta.	Não consta.
(Liu; Shell, 2012)	Baseia-se no <i>Assignment Problem</i> (emparelhamento bipartido).	Não consta.	Não consta.	Utiliza particionamento multinível (METIS) para dividir o problema global de alocação em clusters.	Não consta.
(Oliveira <i>et al.</i> , 2026)	Aplica conceitualmente as regras de HR/HRT.	Compara critérios de desempate processual entre diferentes modalidades.	Aproxima a otimização via algoritmo Gale-Shapley acoplado a heurísticas de desempate.	Não consta.	Realiza experimentos práticos com dados de concorrência do SISU 2023.
Presente trabalho	Utiliza a base matemática clássica de matching estável para alocação.	Modela a indiferença de escolha entre opções e modalidades de cota.	Busca a solução exata por Programação Inteira em componentes locais.	Decompõe o grafo de cursos em clusters via METIS seguido de reconciliação (GS).	Aplica a arquitetura completa para otimizar alocações do SISU em larga escala.

Fonte: Elaborado pelo autor.

4 METODOLOGIA

Este capítulo apresenta a metodologia proposta, cujo fluxo é resumido na Figura 6. A construção do *dataset* é descrita na Seção 4.1, enquanto a modelagem do MAX-HRT é apresentada na Seção 4.2. A decomposição por *clusters*, com uso de *K-Means Constrained* e grafos, é discutida na Seção 4.3. A alocação local por Programação Inteira é detalhada na Seção 4.4, e o tratamento de candidatos de fronteira por *matching* incremental na Seção 4.5. Por fim, a implementação e a avaliação experimental são apresentadas nas Seções 4.6 e 4.7.

Figura 6: Fluxo de procedimentos metodológicos



Fonte: Elaborado pelo autor

4.1 Construção do *Dataset* Experimental

Os experimentos foram conduzidos com os dados da chamada regular do SISU 2023 BRASIL (2026a), correspondentes ao conjunto mais recente disponível no momento da elaboração desta pesquisa. A base é composta por uma planilha com as vagas ofertadas e por um

arquivo CSV contendo os registros de inscrição dos candidatos nos cursos participantes dessa edição.

Dado a planilha de vagas ofertadas BRASIL (2026a), são descartadas as linhas que não apresentam quantidade positiva de vagas. Após essa filtragem, cada registro passa a representar um hospital $(c, m) \in H$, cuja capacidade $q_{c,m}$ é obtida a partir da coluna QT_VAGAS_CONCORRENCIA. O curso c é identificado pela combinação das colunas CO_IES_CURSO, TURNO e CAMPUS, sendo posteriormente associado a um identificador inteiro denominado ID_CURSO.

As modalidades descritas em formato textual são padronizadas em códigos curtos, como AC, LI_EP, LB_PPI, entre outros. As modalidades de bônus são identificadas pelos rótulos B1, B2, etc., enquanto as ações afirmativas próprias são representadas por V1, V2, etc. Na sequência, as vagas dessas ações afirmativas próprias são reincorporadas à ampla concorrência (AC), permitindo a construção de um conjunto único de modalidades elegíveis para cada candidato (Oliveira *et al.*, 2026).

O arquivo CSV da chamada regular apresenta, em cada linha, a inscrição de um candidato em uma opção de curso. Como a identificação dos candidatos está anonimizada, foi realizada uma identificação aproximada a partir da combinação das colunas INSCRITO, INSCRICAO_ENEM e CPF. Em seguida, atribuiu-se um identificador inteiro distinto, denominado ID_CANDIDATO, a cada candidato reconhecido por essa combinação.

A coluna OPCA0 indica a preferência do candidato pelo curso, sendo 1 a primeira opção e 2 a segunda opção. Os candidatos inscritos em modalidade de bônus são tratados como inscritos em AC (Oliveira *et al.*, 2026). Ao final desse processo de tratamento, são gerados dois *dataframes* principais: *df_concorrencia*, contendo as informações de inscrição, modalidade e elegibilidade dos candidatos, e *df_vagas*, contendo os cursos, modalidades e respectivas capacidades disponíveis para a alocação.

4.2 Modelagem do Problema MAX-HRT

Conforme estabelecido na Seção 2.3, o problema de alocação do SISU é modelado como uma instância do *Hospital/Residents with Ties* (HRT), onde candidatos assumem o papel de residentes e pares (curso, modalidade) o de hospitais com capacidade c_j correspondente ao número de vagas ofertadas.

A hipótese central adotada, seguindo Oliveira *et al.* (2026), é que candidatos são indiferentes entre modalidades de um mesmo curso interessados apenas na aprovação no curso

e instituição, independentemente da cota ocupada. Sob essa hipótese, as preferências dos candidatos contêm empates entre os hospitais de um mesmo curso, caracterizando uma instância HRT com empates unilaterais. As preferências dos hospitais são construídas a partir da nota do ENEM e dos critérios de desempate do edital, produzindo ordenações estritas entre candidatos.

O objetivo é encontrar um emparelhamento fracamente estável de cardinalidade máxima o problema MAX-HRT. Como a resolução direta da instância nacional é inviável dado o caráter *NP-hard* do problema e a escala do SISU (mais de 1 milhão de candidatos e 226 mil vagas no SISU 2023 (Oliveira *et al.*, 2026)), este trabalho adota a estratégia de decomposição detalhada na Seção 4.3.

4.3 Estratégia de Decomposição e Clusterização

A instância global do SISU é decomposta em subinstâncias menores por meio do agrupamento de cursos em *clusters* balanceados, permitindo que o modelo de Programação Inteira seja aplicado localmente em cada subinstância. Essa abordagem é motivada pela complexidade *NP-hard* do MAX-HRT, pois a resolução exata de uma instância com centenas de milhares de candidatos é computacionalmente inviável sem decomposição. Duas estratégias de particionamento são investigadas neste trabalho e descritas a seguir.

4.3.1 *K-Means Constrained*

A primeira estratégia utiliza o algoritmo *K-Means Constrained* (Levy-Kramer, 2018), que permite impor limites mínimos e máximos de elementos por grupo. A similaridade entre cursos é definida pelo compartilhamento de candidatos inscritos simultaneamente em ambos, de modo que cursos com alta co-inscrição tendem a pertencer ao mesmo *cluster*. O balanceamento garante que nenhuma subinstância seja grande o suficiente para inviabilizar a solução exata. A construção da matriz de similaridade envolve operações de elevada dimensionalidade e, para viabilizar esse processamento na escala do SISU, utiliza-se paralelismo em GPU (Cano, 2018).

4.3.2 *Particionamento via Biblioteca METIS*

A segunda estratégia representa a co-inscrição entre cursos como um grafo ponderado, conforme formalizado nas Seções 2.4 e 2.4.2, em que cada vértice representa um curso e o peso de cada aresta corresponde ao número de candidatos inscritos simultaneamente nos

dois cursos conectados. O particionamento é realizado pela biblioteca METIS (Karypis; Kumar, 1998), que implementa o algoritmo multinível em três fases, contração progressiva do grafo, particionamento inicial e refinamento iterativo do corte. O objetivo é minimizar o corte ponderado entre os grupos, o que corresponde diretamente a minimizar o número de candidatos de fronteira aqueles cujas duas opções de curso ficam em subinstâncias distintas. O balanceamento dos *clusters* é controlado por um parâmetro de desequilíbrio máximo que impede que qualquer *cluster* ultrapasse o tamanho ideal, garantindo subinstâncias aproximadamente iguais em número de cursos.

4.3.3 Escolha do Número de Clusters

O número de *clusters* K equilibra dois objetivos conflitantes *clusters* menores facilitam a solução exata por IP, mas aumentam o número de candidatos de fronteira e o custo das iterações incrementais. O valor de K é determinado pelo limite máximo de candidatos por *cluster* estabelecido para viabilizar o modelo IP, e seu impacto na qualidade da alocação e no tempo computacional é avaliado experimentalmente na Seção 5.1.2.

4.4 Alocação na sub-instância por Programação Inteira

Na primeira iteração do algoritmo, cada *cluster* K_i é tratado como uma subinstância independente do problema de alocação, resolvida por Programação Inteira, registrando-se o status de otimalidade retornado pelo solver (Kwanashie; Manlove, 2014), implementado em JuMP¹ com o solver Gurobi².

A escolha do IP para a fase inicial se justifica pelo tamanho reduzido de cada *cluster* após a decomposição como o MAX-HRT é NP-hard, a solução exata da instância nacional completa seria inviável, mas torna-se praticável quando aplicada a subinstâncias com apenas uma fração dos candidatos e cursos totais. O modelo recebe como entrada os candidatos atribuídos ao *cluster*, os hospitais associados, as capacidades de cada modalidade e as preferências locais, produzindo uma solução M_i para cada subinstância. A solução global inicial é a união $\mu = \bigcup_i M_i$.

O modelo incorpora a restrição de estabilidade fraca definida na Seção 2.3.1, garantindo que a solução não admita pares bloqueantes. Caso o solver não encontre solução ótima dentro do tempo limite configurado, o algoritmo realiza automaticamente um *fallback* para o

¹ JuMP Contributors: <https://jump.dev/JuMP.jl/stable/>.

² Gurobi Optimizer: <https://www.gurobi.com/>.

Gale-Shapley com critério LCF, assegurando que todos os *clusters* sejam resolvidos mesmo em situações de *timeout*.

Concluída essa etapa, candidatos de fronteira não alocados em sua primeira opção migram para o *cluster* da segunda opção, iniciando o processo incremental descrito na Seção 4.5.

4.5 Tratamento de Candidatos de Fronteira e emparelhamento incremental

Candidatos com duas opções de curso em *clusters* distintos são denominados *candidatos de fronteira*. Na fase inicial, cada um é atribuído ao *cluster* de sua primeira opção. Caso não seja alocado, migra para o *cluster* da segunda opção e participa de uma nova rodada de matching restrita a esse grupo. Como cada candidato migra no máximo uma vez, o algoritmo sempre termina em número finito de iterações.

Para evitar recomputações completas a cada migração, utiliza-se uma variante incremental do Gale-Shapley apenas os candidatos recém-chegados fazem novas propostas, mantendo as alocações já estabilizadas. Somente os *clusters* que receberam novos candidatos são reprocessados.

Nessas iterações incrementais, adota-se o critério LCF (*Least Competitive First*), que ordena as propostas priorizando vagas com menor razão entre candidatos elegíveis e capacidade, favorecendo modalidades com maior risco de ociosidade (Oliveira *et al.*, 2026). Como alternativa avaliada, o ODF (*Observed Deficit First*) direciona candidatos para vagas com maior déficit de ocupação observado em uma alocação de referência (Oliveira *et al.*, 2026).

4.6 Implementação e Ferramentas Utilizadas

4.6.1 Base de Dados do SISU

A construção e o pré-processamento da base de dados utilizada nos experimentos são descritos em detalhes na Seção 4.1. Em síntese, os experimentos foram conduzidos sobre os dados abertos da chamada regular do SISU 2023 (BRASIL, 2026a), composto por 1.073.024 candidatos e 226.349 vagas distribuídas em 6.402 cursos, resultando nos *dataframes* *df_concorrencencia* e *df_vagas* utilizados como entrada pelos algoritmos implementados.

4.6.2 Ambiente Computacional e Ferramentas

Os algoritmos propostos foram implementados em Julia e em Python, sendo a análise de dados realizada exclusivamente em Python 3.12.10. A manipulação dos dados utilizou Pandas³ e NumPy⁴, com SciPy⁵ para processamento de matrizes esparsas. O agrupamento de cursos foi realizado com a biblioteca *K-Means Constrained*⁶. Os modelos de Programação Inteira foram formulados em JuMP⁷ e resolvidos pelo solver Gurobi⁸, escolhido por sua licença acadêmica gratuita e desempenho consolidado em problemas de otimização combinatória de larga escala.

Os experimentos foram executados em um notebook Lenovo LOQ 15IAX9E com processador Intel Core i5-12450HX (12 núcleos), 16 GB de RAM e GPU NVIDIA GeForce RTX 3050 6GB, sob Ubuntu 24.04.4 LTS. A GPU foi utilizada opcionalmente para acelerar a construção da matriz de co-inscrição em instâncias de maior porte.

4.7 Avaliação Experimental

Os experimentos foram conduzidos sobre os dados reais do SISU 2023, composto por 1.073.024 candidatos e 226.349 vagas distribuídas em 6.402 cursos. A avaliação tem dois objetivos principais: verificar se a decomposição proposta produz alocações de qualidade equivalente aos algoritmos de referência, e aferir a viabilidade computacional do modelo IP em subinstâncias de escala nacional.

4.7.1 Algoritmos Avaliados

Foram avaliados quatro algoritmos sobre a mesma instância do SISU 2023. O ASISU serve como *baseline*, correspondendo à aproximação do processo oficial sem decomposição. LCF e ODF são heurísticas determinísticas de desempate avaliadas em Oliveira *et al.* (2026) como referências comparativas. O quarto é o algoritmo proposto neste trabalho, IP+GS Incremental, que combina solução exata por Programação Inteira com Gale-Shapley incremental sobre *clusters* balanceados.

³ Pandas: <https://pandas.pydata.org/>.

⁴ NumPy: <https://numpy.org/>.

⁵ SciPy: <https://scipy.org/>.

⁶ K-Means Constrained: <https://github.com/joshlk/k-means-constrained>.

⁷ JuMP Contributors: <https://jump.dev/JuMP.jl/stable/>.

⁸ Gurobi Optimizer: <https://www.gurobi.com/>.

4.7.2 ASISU — *Baseline*

O ASISU é executado sobre a instância global sem decomposição, aplicando o Gale-Shapley com a hierarquia de modalidades do edital do SISU 2023 para resolver os empates (Oliveira *et al.*, 2026). Por operar sobre a instância completa, serve como referência direta para avaliar o impacto da decomposição em *clusters* na cardinalidade da alocação e no tempo de execução.

4.7.3 *Least Competitive First (LCF)*

O LCF é executado sobre a instância global, resolvendo os empates por ordem crescente da razão candidatos elegíveis/vagas disponíveis por modalidade (Oliveira *et al.*, 2026). Nos experimentos, serve para avaliar se a estratégia de decomposição proposta supera a melhor heurística determinística em termos de candidatos alocados.

4.7.4 *Observed Deficit First (ODF)*

O ODF é executado em duas etapas sobre a instância global, primeiro com o critério ASISU para obter uma alocação de referência, depois com as listas reordenadas pelo déficit observado de cada modalidade (Oliveira *et al.*, 2026). Nos experimentos, permite avaliar se a informação endógena da primeira execução produz ganhos comparáveis aos obtidos pela decomposição proposta.

4.7.5 *IP+GS Incremental — Algoritmo Proposto*

O algoritmo proposto decompõe a instância nacional em K *clusters* de cursos formados por similaridade de co-inscrição, resolve cada *cluster* localmente com o modelo de Programação Inteira exato de Kwanashie e Manlove (2014), e trata os candidatos de fronteira por meio de Gale-Shapley incremental com critério LCF. Essa abordagem se justifica pelo fato de que o MAX-HRT é NP-*hard* na instância completa, mas torna-se computacionalmente tratável quando decomposto em subinstâncias de tamanho reduzido e balanceado.

Fonte: Elaborado pelo autor

Algoritmo 5: Algoritmo IP+GS Incremental

Entrada: Candidatos R , Hospitais H , *Clusters* $K_1, \dots, K_p$

Saída: Emparelhamento fracamente estável μ

// Fase 1: Alocação local por Programação Inteira

foreach *cluster* K_i **do**

 Construa subinstância S_i com candidatos e hospitais de K_i ;

$M_i \leftarrow \text{IP}(S_i)$;

end

$\mu \leftarrow \bigcup_i M_i$;

$Q \leftarrow \{r \in R : r \text{ é de fronteira, estado}(r) = \text{PRIMEIRA_OPÇÃO}, r \notin \mu\}$;

// Fase 2: Loop incremental para candidatos de fronteira

while $Q \neq \emptyset$ **do**

$Dirty \leftarrow \emptyset$;

foreach $r \in Q$ **do**

 Migre r para $K_{\text{cluster}(c_2(r))}$;

$\text{estado}(r) \leftarrow \text{SEGUNDA_OPÇÃO}$;

 Adicione $\text{cluster}(c_2(r))$ a $Dirty$;

end

foreach *cluster* $K_i \in Dirty$ **do**

$M_i \leftarrow \text{GS_LCF_Incremental}(K_i, M_i, \text{novos}_i)$;

end

$\mu \leftarrow \bigcup_i M_i$;

$Q \leftarrow \{r \in R : r \text{ é de fronteira, estado}(r) = \text{PRIMEIRA_OPÇÃO}, r \notin \mu\}$;

end

return μ ;

4.7.6 Métricas de Avaliação

A qualidade de cada solução é medida pelo número total de candidatos alocados e pela taxa de ocupação das vagas. A estabilidade do emparelhamento é verificada pela ausência de pares bloqueantes (Gale; Shapley, 1962). Para o algoritmo proposto, registram-se adicionalmente métricas como número de candidatos de fronteira, quantidade de migrações entre *clusters* e total de iterações do loop incremental até a convergência.

4.7.7 Artefatos Gerados

Ao término da execução, o sistema produz automaticamente os seguintes arquivos:

- `clusters_cursos.csv`: um registro por curso com o *cluster* atribuído, total de candidatos e separação entre candidatos internos e de fronteira;
- `cluster_sizes.csv`: resumo por *cluster* com número de cursos, candidatos, internos, fronteira e vagas disponíveis;

- `alocacoes_finais.csv`: alocação final do algoritmo proposto, com candidato, curso, modalidade, modalidade original de inscrição e nota;
- `comparacao_algoritmos.csv`: tabela consolidada dos três algoritmos com métricas de candidatos alocados, taxa de ocupação, tempo de execução e instabilidades;
- `metricas_tcc.csv` e `metricas_tcc.txt`: detalhes internos do algoritmo proposto, incluindo tempos por etapa, número de migrações, iterações do loop e coeficiente de variação dos *clusters*.

Os gráficos gerados incluem, distribuição de cursos por *cluster*; tempo de execução por etapa; convergência da fila de fronteira ao longo das iterações; impacto do loop incremental no total de alocados; painel comparativo consolidando candidatos alocados, tempo de execução e instabilidades entre os quatro algoritmos; e um gráfico analisando o comportamento dos candidatos de fronteira em função do número de *clusters* K .

4.8 Cronograma

O quadro abaixo apresenta o cronograma de execução das atividades desenvolvidas neste trabalho, contemplando desde a construção da base de dados até a implementação e avaliação experimental.

quadro 02: Cronograma de execução das atividades.

Etapa	Mar.	Abr.	Mai.	Jun.	Jul.	Ago.	Set.	Out.	Nov.	Dez.
Construção do Dataset Experimental	x	x								
Modelagem do Problema MAX-HRT		x	x							
Estratégia de Decomposição e Clusterização			x	x						
Alocação Local por Programação Inteira				x	x					
Tratamento de Candidatos de Fronteira e Matching Incremental			x	x	x					
Implementação e Ferramentas Utilizadas			x	x	x	x				
Avaliação Experimental			x	x	x					
Escrita e revisão do TCC 2						x	x	x	x	
Defesa do TCC 2										x

Fonte: Elaborado pelo autor.

5 RESULTADOS PRELIMINARES

Este capítulo dedica-se à análise exploratória da base de dados do SISU, cujas características justificam a estratégia de decomposição proposta neste trabalho. A exposição inicia na Seção 5.1, com a caracterização da rede de concorrência e de seus principais *hubs*; em seguida, a Seção 5.1.2 detalha as conexões mais relevantes entre cursos, evidenciando os padrões de co-inscrição que dificultam o particionamento. A Seção 5.1.3 avalia o efeito dos cursos de Medicina sobre os candidatos de fronteira, enquanto a Seção 5.2 relaciona demanda, oferta e número de *clusters*. A Seção 5.3 compara a cluterização com os dois tipos de cluterização abordados. Na sequência, a Seção 5.4 apresenta os resultados de ocupação obtidos pelo modelo IP e a Seção 5.5 sintetiza a viabilidade da abordagem para o SISU em escala nacional.

5.1 Análise Exploratória dos Dados

A base analisada reúne 1.073.024 candidatos, 226.390 vagas e 6.402 cursos do SISU. Esse volume evidencia uma rede de concorrência densa, com fortes interdependências entre cursos e presença de *hubs*, que concentram grande parte das conexões e tornam o particionamento do grafo mais desafiador.

5.1.1 Análise de Hubs e Centralidade da Rede de Concorrência

Para mapear as relações entre os cursos, o SISU foi representado por uma matriz de adjacência ponderada de dimensões 6.402×6.402 . Neste trabalho, uma conexão corresponde a um candidato que se inscreve simultaneamente em dois cursos distintos, contribuindo com uma unidade de peso para a aresta entre esse par de cursos. Assim, a matriz pode ser interpretada como a projeção ponderada de um multigrafo de co-inscrição cada par de cursos possui peso igual ao número de candidatos em comum. A soma das linhas mostrou que poucos cursos concentram muitas conexões ponderadas, enquanto a maioria possui baixa interação com os demais. Esses cursos mais conectados funcionam como *hubs* da rede de concorrência. O Quadro 2 apresenta os 15 principais *hubs* identificados¹.

Os dados provam que a complexidade estrutural do SISU é causada pelo curso de Medicina, que ocupa 11 das 15 maiores centralidades. O comportamento estratégico desses

¹ O código-fonte em Python para a tradução automatizada dos identificadores de curso e análise de metadados associados está publicamente disponível em: <https://github.com/Naliat/TCC1/blob/main/curso.py>

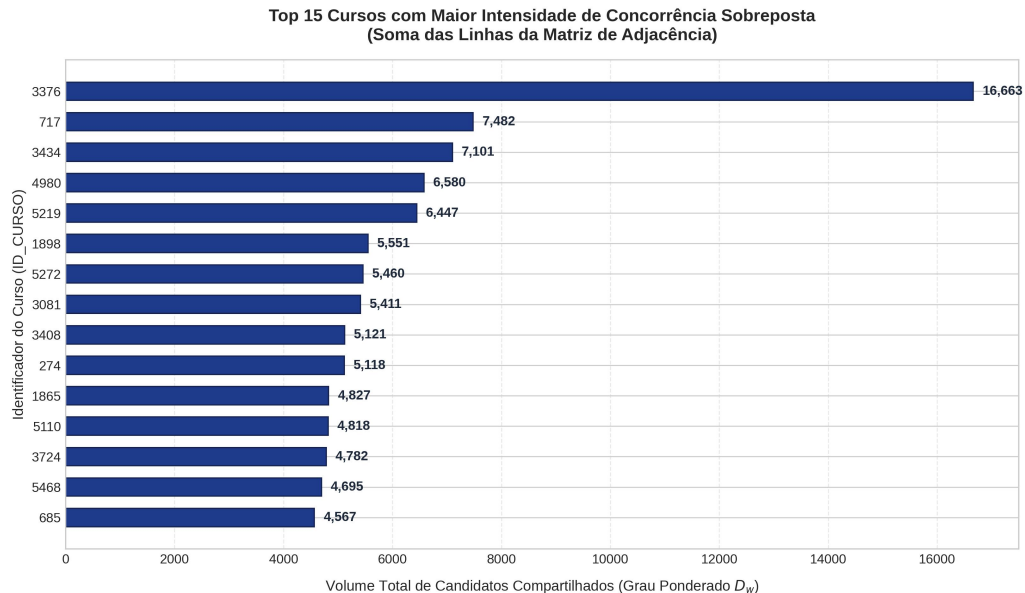
ID	Curso	Instituição	Campus	UF	Conexões
3376	Medicina	UFMG	Campus Saúde	MG	16.663
717	Medicina	UFC	Campus Fortaleza	CE	7.482
3434	Medicina	UFSCAR	Unidade SEDE	SP	7.101
4980	Medicina	UFG	Reg. Goiânia - Colemar	GO	6.580
5219	Medicina	UFRJ	Cidade Universitária	RJ	6.447
1898	Direito	UNIFESP	Campus Osasco	SP	5.551
5272	Medicina	FURG	Campus da Saúde	RS	5.460
3081	Medicina	UFAL	Campus A. C. Simões	AL	5.411
3408	Psicologia	UFMG	Campus Pampulha	MG	5.121
274	Medicina	UFOB	Campus Reitor Edgard Santos	BA	5.118
1865	Medicina	ESCS	Brasília - Asa Norte	DF	4.827
5110	Psicologia	UFRJ	Praia Vermelha	RJ	4.818
3724	Medicina	UFF	Faculdade de Medicina	RJ	4.782
5468	ADS*	IFSP	IFSP - Campus São Paulo	SP	4.695
685	Medicina	UFERSA	Unidade SEDE	RN	4.467

Fonte: Autor próprio. *Análise e Desenvolvimento de Sistemas.

quadro 2: Mapeamento dos 15 principais *hubs* de concorrência cruzada do SISU 2023.

candidatos que tendem a fixar o curso em ambas as opções, variando apenas a instituição ou o *campus*. Essa dinâmica amarra o grafo nacional de forma indissociável, conforme ilustrado na Figura 7.

Figura 7: Intensidade de concorrência sobreposta (Soma das Linhas da Matriz de Adjacência) para o Top 15.



Fonte: Elaboração própria.

A presença desses *super-hubs* dificulta o uso de um particionamento puramente topológico, como o METIS. Se todas as conexões de um curso muito central, como Medicina, fossem mantidas dentro de um mesmo grupo, muitos cursos de diferentes regiões acabariam reunidos

em um único bloco excessivamente grande, aumentando o custo computacional e potencialmente inviabilizando a etapa de Programação Inteira. Por essa razão, a decomposição inevitavelmente produz candidatos de fronteira, cujas duas opções de curso pertencem a componentes distintos do grafo particionado. Esses candidatos não podem ser simplesmente descartados ou tratados de forma aproximada a fase de Gale-Shapley incremental, é essencial não apenas quando o volume de candidatos de fronteira é alto, mas mesmo quando esse volume é pequeno, pois é ela que garante a estabilidade da solução final sem o tratamento incremental, a alocação obtida exclusivamente pelas soluções locais de cada componente não constituiria um emparelhamento estável para a instância completa do SISU.

5.1.2 Cursos e suas principais conexões

Para compreender a topologia de concorrência do SISU, analisamos a densidade das arestas (co-inscrições) incidentes sobre três *hubs* estratégicos Medicina na UFMG (ID 3376), Medicina na UFC (ID 717) e Medicina na UFSCar (ID 3434)². A Tabela 4 quantifica os fluxos de candidatos que compartilham a mesma preferência por essas instituições, detalhando o curso e o respectivo campus.

Tabela 4: Top 5 cursos com maior adjacência ponderada discriminada por *hub*, IES e campus.

Hub de Origem	Pos.	Curso Relacionado	IES	Campus	Candidatos em Comum
Medicina (UFMG - Saúde)	#1	Medicina	UFSCar	Unidade SEDE	1.221
	#2	Medicina	UFC	Fortaleza	899
	#3	Medicina	UFRJ	Cidade Universitária	750
	#4	Medicina	UFG	Reg. Goiânia - Colemar Natal	511
	#5	Medicina	UFOP	Campus Morro do Cruzeiro	492
Medicina (UFC - Fortaleza)	#1	Medicina	UFMG	Campus Saúde	899
	#2	Medicina	UFC	Campus UFC Sobral	862
	#3	Medicina	UFCA	Campus Barbalha	539
	#4	Enfermagem	UFC	Fortaleza	369
	#5	Medicina	UFAL	Campus A. C. Simões	297
Medicina (UFSCar - Sede)	#1	Medicina	UFMG	Campus Saúde	1.221
	#2	Enfermagem	UFSCar	Unidade SEDE	465
	#3	Medicina	UFRJ	Cidade Universitária	334
	#4	Medicina	UFPR	Coordenação de Medicina	255
	#5	Medicina	UNIFAL	Campus Sede Alfenas	242

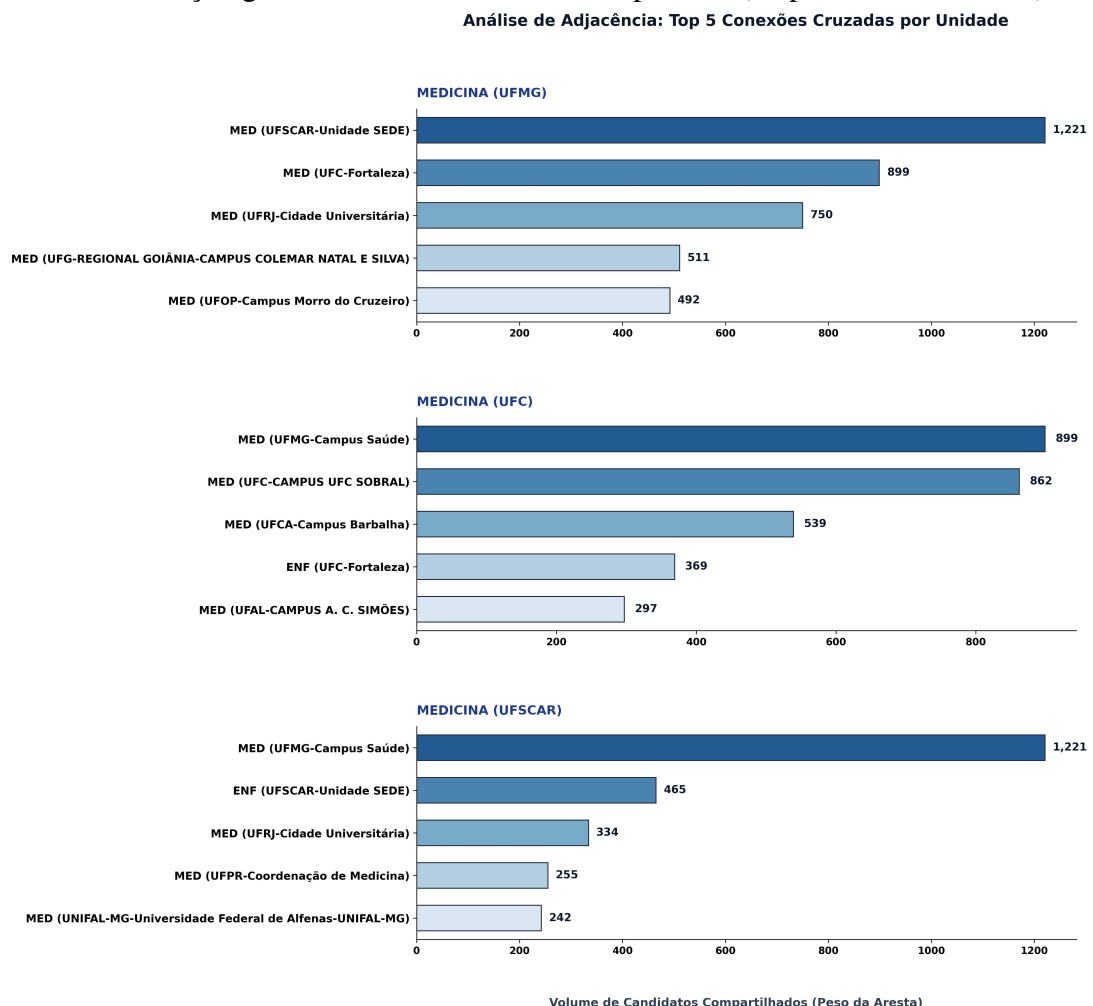
Os dados mostram dois padrões principais de escolha. O primeiro é a migração representada pela conexão entre UFMG e UFSCar, com 1.221 candidatos em comum entre o

² Código disponível em: <https://github.com/Naliat/TCC1/blob/main/graficos/cursoRelacionado.py>.

Campus Saúde (MG) e a Unidade SEDE (SP). Essa ligação é forte e conecta regiões distantes, tornando o particionamento do grafo mais difícil.

Adicionalmente, observa-se o fenômeno da ancoragem institucional, onde cursos como Enfermagem na UFC (Fortaleza) e na UFSCar (Unidade SEDE) aparecem como estratégias de segurança para área da saúde. Esses cursos possuem alta correlação com os *hubs* principais, servindo como uma alternativa tática para candidatos que, visando a mesma universidade, buscam otimizar a probabilidade de ingresso. Há também uma forte ancoragem regional geográfica, evidenciada pelos 862 candidatos em comum entre UFC Fortaleza e o Campus UFC Sobral. A Figura 8 ilustra visualmente a disparidade de volume dessas conexões de forma empilhada.

Figura 8: Distribuição gráfica das conexões cruzadas por *hub* (empilhamento vertical).



Fonte: autor próprio

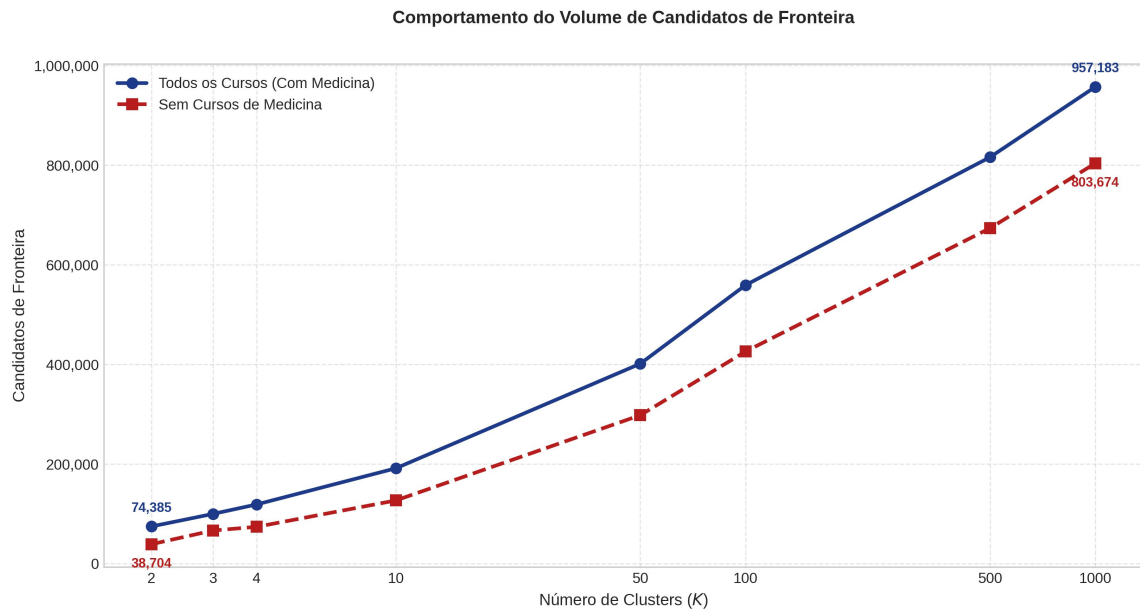
Essas conexões mostram que uma segmentação puramente topológica não é suficiente para lidar com a complexidade do SISU. Ao agrupar cursos muito conectados, o particionamento multinível pode formar blocos excessivamente densos, aumentando a necessidade de tratar

candidatos de fronteira por meio de um algoritmo incremental de *matching*, conforme discutido na próxima seção.

5.1.3 Análise de Candidatos de Fronteira com e sem Medicina via Metis

Para quantificar o impacto dos cursos de Medicina na formação de candidatos de fronteira, conduziu-se um experimento sistemático sobre os dados reais do SISU 2023³. Dois cenários foram avaliados para valores de $K \in \{2, 3, 4, 10, 50, 100, 500, 1000\}$ o primeiro considera todos os 6.402 cursos disponíveis o segundo exclui os 88 cursos de Medicina, correspondentes a 1,37% do total. Os resultados são apresentados na Figura 9 e na Tabela 5.

Figura 9: Candidatos de fronteira em função de K com e sem Medicina



Fonte: Autor próprio.

Os resultados revelam um padrão consistente e com implicações diretas para a estratégia de decomposição adotada. Para $K = 2$, remover os 88 cursos de Medicina reduz a fronteira em 48% de 74.385 para 38.704 candidatos. Esse efeito, porém, diminui progressivamente com o aumento de K , chegando a apenas 16% de redução em $K = 1000$, o que indica que o impacto de Medicina se dilui à medida que os *clusters* se tornam menores e os demais cursos também passam a se separar. Mais revelador ainda é o fato de que 989.791 dos 1.073.024 candidatos ou seja, 92,2% possuem duas opções em cursos distintos, o que significa que qualquer decomposição, independentemente do algoritmo utilizado, produzirá um volume expressivo de candidatos de

³ Código disponível em: https://github.com/Naliat/TCC1/blob/main/graficos/fronteira_metis.py.

Tabela 5: Candidatos de fronteira por valor de K

K	Com Medicina	Sem Medicina	Redução (%)
2	74.385	38.704	48,0
3	99.560	66.236	33,5
4	118.635	73.641	37,9
10	191.177	126.615	33,8
50	401.222	297.770	25,8
100	559.067	425.791	23,8
500	815.955	673.321	17,5
1000	957.183	803.674	16,0

Fonte: Autor próprio.

fronteira.

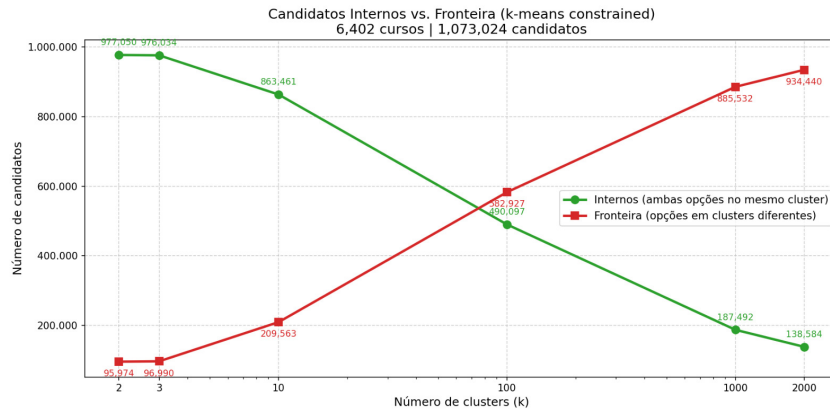
Esses resultados demonstram que, por mais que seja possível identificar e tratar os cursos mais conectados individualmente, o SISU, por sua própria natureza, forma um grafo de co-inscrição denso e altamente interconectado, tornando a fronteira uma característica estrutural do problema e não uma limitação do método de particionamento adotado. No mais, para fins de demonstração da formulação *MAX-HRT* em programação inteira, vamos usar o modelo do SISU completo sem tratamento adicional para retirar esses cursos super conectados como o de Medicina.

5.2 Análise de Demanda, Oferta e Clusterização via k-means-constrained

A relação entre candidatos e vagas varia bastante entre os cursos, principalmente naqueles com maior procura. Como o SISU envolve muitos cursos, instituições, vagas e modalidades de concorrência, resolver todo o problema de uma só vez por Programação Inteira exigiria um esforço computacional muito alto.

A Figura 10 apresenta o comportamento dos candidatos de fronteira. Observa-se que o aumento no número de *clusters* também eleva a quantidade de candidatos de fronteira. Por outro lado, essa divisão reduz o tamanho das subinstâncias, facilitando a busca realizada pelo modelo de Programação Inteira para o *MAX-HRT*.

Figura 10: Comportamento dos candidatos de fronteira em função do número de clusters (K).



Fonte: Elaboração própria.

Nos testes visualizado na 10 clusterização com o algoritmo *k-means constrained*, observou-se um custo computacional elevado, tanto pelo tempo de execução quanto pelo consumo de memória RAM. Embora, em alguns valores maiores de K , o método tenha gerado menos candidatos de fronteira, seu desempenho geral mostrou-se pouco adequado para a escala do SISU. Por esse motivo, nos experimentos seguintes, optou-se pelo algoritmo baseado em grafos *k-way partitioning* da biblioteca METIS.

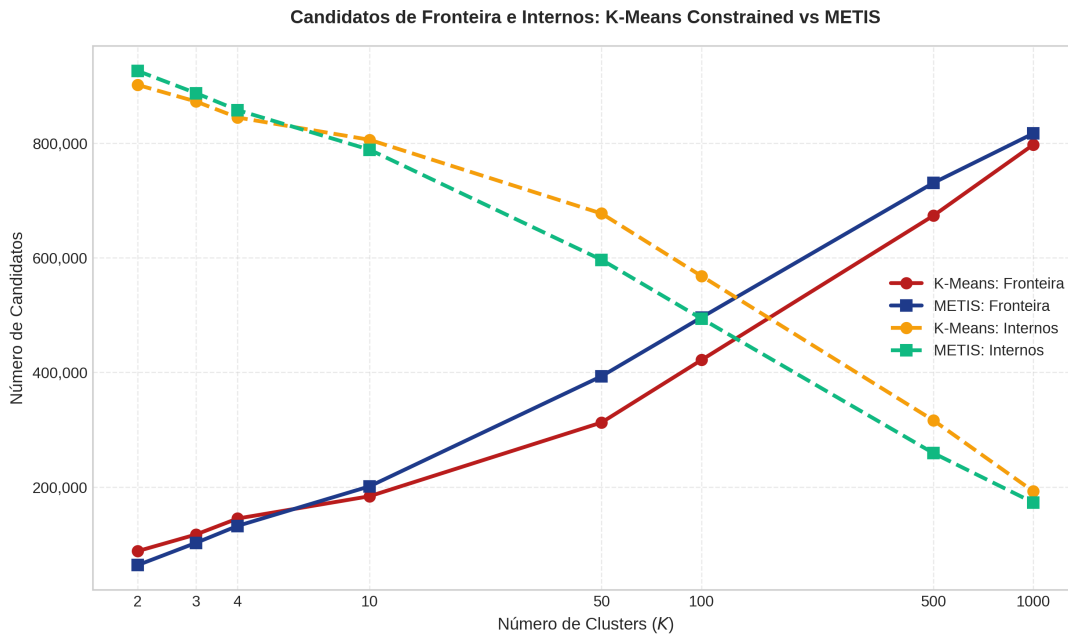
5.3 Comparação entre *K-Means Constrained* e METIS

Para avaliar empiricamente as duas estratégias de decomposição descritas nas Seções 4.3.1 e 4.3.2, conduziu-se um experimento sobre os dados reais do SISU 2023, composto por 1.073.024 candidatos e 6.402 cursos. Dos candidatos com duas opções de curso distintas, 989.791 foram considerados na análise, variando o número de *clusters* $K \in \{2, 3, 4, 10, 50, 100, 500, 1000\}$. Para cada valor de K , mediu-se o número de candidatos internos cujas duas opções permanecem no mesmo *cluster* e de candidatos de fronteira cujas opções ficam em *clusters* distintos gerados por cada estratégia. Os resultados são apresentados na Tabela 6 e na Figura 11.

Tabela 6: Candidatos de fronteira e internos por valor de K

K	K -Means Fronteira	K -Means Internos	METIS Fronteira	METIS Internos
2	88.265	901.526	63.857	925.934
3	117.335	872.456	102.744	887.047
4	145.130	844.661	132.266	857.525
10	184.056	805.735	201.319	788.472
50	312.669	677.122	393.367	596.424
100	421.678	568.113	495.896	493.895
500	673.481	316.310	730.601	259.190
1000	797.261	192.530	816.766	173.025

Fonte: Autor próprio.

Figura 11: Candidatos de fronteira e internos: K -Means Constrained vs METIS

Fonte: Autor próprio.

Os resultados mostram que, para valores pequenos de K ($K \leq 4$), o METIS gera consistentemente menos candidatos de fronteira que o K -Means Constrained. Em $K = 2$, por exemplo, o METIS produz 63.857 candidatos de fronteira contra 88.265 do K -Means, uma diferença de aproximadamente 28%.

A partir de $K = 10$, esse padrão se inverte: o METIS passa a gerar mais candidatos de fronteira que o K -Means Constrained, e essa diferença se amplia conforme K aumenta. Em $K = 100$, o METIS produz 495.896 candidatos de fronteira contra 421.678 do K -Means; em $K = 1000$, a diferença permanece na mesma ordem de grandeza, com 816.766 contra 797.261. Esse resultado deve ser interpretado como uma evidência empírica do experimento, e não como

uma garantia teórica de superioridade do *K-Means*. Embora o METIS tenha como objetivo direto reduzir o corte entre partições, essa otimização é realizada por heurísticas multinível, sem certificado de otimalidade global. O *K-Means Constrained*, por outro lado, não otimiza diretamente o número de candidatos de fronteira; seu objetivo está relacionado à formação de grupos balanceados no espaço de atributos utilizado. Ainda assim, nos valores mais altos de K analisados, o balanceamento imposto pelo *K-Means* produziu empiricamente menos candidatos de fronteira que o METIS. Esse comportamento sugere que, nesta instância do SISU, a qualidade das soluções heurísticas do METIS pode se degradar mais rapidamente, para K grande, do que o efeito indireto de balanceamento produzido pelo *K-Means Constrained*.

Em ambas as estratégias, observa-se que a curva de candidatos internos é a imagem espelhada da curva de fronteira, como esperado, dado que a soma das duas métricas é constante e igual ao total de 989.791 candidatos analisados. Para $K = 1000$, apenas 173.025 candidatos (METIS) e 192.530 candidatos (*K-Means*) permanecem internos, evidenciando que, na escala de decomposição necessária para viabilizar o modelo de Programação Inteira por *cluster*, a maior parte dos candidatos com duas opções de curso será tratada como fronteira por qualquer uma das duas estratégias, reforçando a relevância do mecanismo de migração incremental descrito na Seção 4.5.

5.4 Desempenho Geral do Modelo *MAX-HRT*

Para avaliar o desempenho do modelo *MAX-HRT*⁴, baseado em Kwanashie e Manlove (2014), foram realizados experimentos computacionais em Julia, com o pacote JuMP e o *solver* Gurobi. O estudo de caso considerou o mercado nacional do curso de Ciência da Computação, utilizando microdados reais do SISU.

Todas as execuções foram concluídas com sucesso. O modelo preencheu todas as 495 vagas disponíveis na instância analisada, alcançando taxa de ocupação de 100%. A Tabela 7 apresenta os principais resultados de desempenho e alocação.

Além da ocupação das vagas, também foi avaliada a qualidade da formulação matemática. Para isso, analisou-se a relaxação linear contínua do modelo, que permite verificar se a formulação produz limites próximos da solução inteira. A Tabela 8 mostra o tamanho do modelo gerado e os principais valores retornados pelo solucionador.

⁴ Código-fonte da solução disponível em: https://github.com/Naliat/TCC1/blob/main/grafos/hrt_model.jl.

Tabela 7: Métricas de desempenho computacional e alocação global

Indicador	Valor
Candidatos analisados (N_{CAND})	42.531
Unidades ofertadas / hospitais (N_{HOSP})	102
Total de vagas ofertadas	495
Total de vagas preenchidas	495
Taxa de ocupação	100,0%
Tempo de resolução em (s)	16,8906
Pico de consumo de memória RAM (MiB)	2.967,59

Tabela 8: Validação estrutural do modelo *IP (MAX-HRT)*

Métrica do modelo <i>IP</i>	Valor
Número de variáveis binárias ($x_{i,j}$)	48.165
Número de restrições lineares	138.963
Solução inteira encontrada (<i>Incumbent</i>)	495,0
Limite superior contínuo (<i>Best Bound</i>)	495,0
<i>Gap</i> de otimização relativo	0,0%
Nós explorados na árvore (<i>Branch-and-Bound</i>)	0
Iterações do algoritmo de remanejamento	1

Fonte: Elaborado pelo autor.

5.4.1 Análise dos Resultados Computacionais

Os resultados indicam dois pontos importantes. O primeiro é que a formulação de Kwanashie e Manlove (2014) se mostrou forte para a instância analisada. O *Gap* de otimização foi de 0,0% já no nó inicial da busca, indicando que a relaxação linear encontrou o mesmo valor da solução inteira. Por isso, o solucionador não precisou explorar novos nós na árvore de *Branch-and-Bound* (Nós = 0). O segundo ponto é que o remanejamento de vagas terminou logo na primeira iteração. Não houve vagas remanejadas, pois a demanda pelo curso de Ciência da Computação foi muito maior do que a oferta 42.531 candidatos disputaram apenas 495 vagas. Com tantos candidatos para poucas vagas, todas as modalidades foram preenchidas rapidamente. Esse resultado mostra que, em cursos com alta demanda, o modelo consegue preencher as vagas de forma direta e sem ociosidade. Assim, a formulação se mostrou adequada para instâncias concentradas em cursos muito concorridos.

5.5 Discussão e Viabilidade Computacional para o SISU

Os resultados experimentais obtidos confirmam a viabilidade computacional do modelo *IP* quando aplicado a subinstâncias isoladas ou controladas. No estudo de caso preliminar de Ciência da Computação, o tempo total de *clock* do *solver* foi de apenas 16,89 segundos.

Contudo, a transição desse modelo para uma aplicação diretamente sobre o SISU completo, composto por mais de um milhão de candidatos, aproximadamente 226 mil vagas físicas e 6.400 opções de cursos conectados por escolhas que podem ajudar na criação de candidatos de fronteira impõe barreiras matemáticas e de infraestrutura severas.

O primeiro grande gargalo está relacionado ao crescimento do número de combinações do modelo. O *MAX-HRT*, na versão com estabilidade fraca de Kwanashie e Manlove (2014), exige restrições para cada par candidato-curso elegível. No SISU completo, esse número cresce muito, principalmente por causa das diferentes modalidades de concorrência. Com isso, aumentam tanto as variáveis binárias ($x_{i,j}$) quanto as restrições de estabilidade, elevando o consumo de memória RAM. Em alguns testes realizados, essa limitação apareceu antes mesmo do início da otimização, durante a preparação do modelo pelo resolvidor (*Presolve*). Portanto, para aplicar a abordagem em instâncias maiores, será necessário contornar esse problema, seja com algoritmos mais eficientes no uso de memória, seja com o aumento da capacidade de memória RAM do *hardware* utilizado.

Para reduzir esse problema, a solução mais adequada é dividir o SISU em *clusters* menores e bem balanceados. Essa divisão evita que um único grupo concentre candidatos demais, o que poderia manter o alto consumo de memória e aumentar o tempo de execução. Assim, a clusterização ajuda a transformar o problema completo em partes menores, mais fáceis de processar e mais compatíveis com a capacidade do *hardware* disponível.

REFERÊNCIAS

- ANDREOTTI, E.; EDELMANN, D.; GUGLIELMI, N.; LUBICH, C. Constrained graph partitioning via matrix differential equations. **SIAM Journal on Matrix Analysis and Applications**, v. 40, n. 1, p. 1–22, 2019. Disponível em: <https://doi.org/10.1137/17M1160987>.
- BONDY, J. A.; MURTY, U. S. R. **Graph Theory with Applications**. London, England: Macmillan Press, 1976. Disponível em: <http://www.zib.de/groetschel/teaching/WS1314/BondyMurtyGTWA.pdf>.
- BRASIL. **Lei nº 12.711, de 29 de agosto de 2012**: Dispõe sobre o ingresso nas universidades federais e nas instituições federais de ensino técnico de nível médio e dá outras providências. 2012. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2011-2014/2012/lei/l12711.htm.
- BRASIL. **Como funciona a classificação dos candidatos no Sisu?** Brasília, DF: MEC, 2023. Disponível em: <https://www.gov.br/mec/pt-br/aceso-a-informacao/perguntas-frequentes/sisu/remanejamento-de-vagas/como-funciona-a-classificacao-dos>. Acesso em: 16 jun. 2026.
- BRASIL. **Lei nº 14.723, de 13 de novembro de 2023**: Altera a lei nº 12.711, de 29 de agosto de 2012. Brasília, DF: [S. n.], 2023. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2023-2026/2023/lei/l14723.htm. Acesso em: 21 abr. 2026.
- BRASIL. **Como funciona a classificação dos candidatos e o remanejamento de vagas?** 2025. Disponível em: <https://www.gov.br/mec/pt-br/aceso-a-informacao/perguntas-frequentes/sisu/remanejamento-de-vagas/como-funciona-a-classificacao-dos>. Acesso em: 14 maio 2026.
- BRASIL. **Quais os critérios de desempate?** 2025. Disponível em: <https://www.gov.br/mec/pt-br/aceso-a-informacao/perguntas-frequentes/programa-universidade-para-todos-prouni/processo-seletivo/quais-os-criterios-de-desempate>. Acesso em: 13 maio 2026.
- BRASIL. **Quais são as modalidades de concorrência disponíveis no Sisu?** 2025. Disponível em: <https://www.gov.br/mec/pt-br/aceso-a-informacao/perguntas-frequentes/sisu/modalidades-de-vagas/quais-sao-as-modalidades-de>. Acesso em: 14 maio 2026.
- BRASIL. **Dados Abertos do SISU**. 2026. Acesso em: 19 maio 2026. Disponível em: <https://dadosabertos.mec.gov.br/sisu>.
- BRASIL. **Dúvidas Frequentes: Sisu - Nota de Corte**. 2026. Disponível em: <https://accessunico.mec.gov.br/sisu/duvidas#nota-de-corte>. Acesso em: 14 maio 2026.
- CANO, A. A survey on graphic processing unit computing for large-scale data mining. **WIREs Data Mining and Knowledge Discovery**, Hoboken, NJ, v. 8, n. 1, p. e1232, jan 2018. Disponível em: <https://onlinelibrary.wiley.com/doi/abs/10.1002/widm.1232>.
- DING, C. H. Q.; HE, X.; ZHA, H.; GU, M.; SIMON, H. D. A min-max cut algorithm for graph partitioning and data clustering. In: **Proceedings 2001 IEEE International Conference on Data Mining (ICDM)**. [S. n.], 2001. p. 107–114. Disponível em: <https://doi.org/10.1109/ICDM.2001.989507>.
- FELDMANN, A.; FOSCHINI, L. Balanced partitions of trees and applications. **Algorithmica**, 2012. Disponível em: <https://hal.science/hal-00678179>.

FENOALTEA, E. M.; BAYBUSINOV, I. B.; ZHAO, J.; ZHOU, L.; ZHANG, Y.-C. The stable marriage problem: An interdisciplinary review from the physicist's perspective. **Physics Reports**, v. 917, p. 1–79, 2021. Disponível em: <https://doi.org/10.1016/j.physrep.2021.03.001>.

GALE, D.; SHAPLEY, L. S. College admissions and the stability of marriage. **The American Mathematical Monthly**, v. 69, n. 1, p. 9–15, 1962. Disponível em: <https://doi.org/10.2307/2312726>.

GLADKOV, L. A. Topics in graph theory. **AMS/MAA Textbooks**, 2018. Disponível em: <https://api.semanticscholar.org/CorpusID:29773241>.

GUBITOSO, A. G.; LOPES, V. C. others. O algoritmo de gale-shapley. In: _____. **Alocações, estabilidade e otimização: uma introdução passo a passo**. São Bernardo do Campo: Editora UFABC, 2017. cap. 1, p. 5–22. Acesso em: 2 jun. 2026. Disponível em: <https://doi.org/10.7476/9788568576823.0002>.

IRVING, R. W.; MANLOVE, D. F.; SCOTT, S. The hospitals/residents problem with ties. In: **Proceedings of the 7th Scandinavian Workshop on Algorithm Theory (SWAT 2000)**. Bergen, Norway: Springer-Verlag, 2000. (Lecture Notes in Computer Science, v. 1851), p. 259–271. Disponível em: https://doi.org/10.1007/3-540-44985-X_24.

KARYPIS, G.; KUMAR, V. A fast and high-quality multilevel scheme for partitioning irregular graphs. **SIAM Journal on Scientific Computing**, SIAM, v. 20, n. 1, p. 359–392, 1998. Disponível em: <https://doi.org/10.1137/S1064827595287997>.

KWANASHIE, A.; MANLOVE, D. F. An integer programming approach to the hospitals/residents problem with ties. In: HUISMAN, D.; LOUWERSE, I.; WAGELMANS, A. P. M. (Ed.). **Operations Research Proceedings 2013**. Cham: Springer International Publishing, 2014. p. 263–269. ISBN 978-3-319-07001-8. Disponível em: https://doi.org/10.1007/978-3-319-07001-8_36.

LEVY-KRAMER, J. **k-means-constrained**. 2018. Disponível em: <https://github.com/joshlk/k-means-constrained>.

LIU, L.; SHELL, D. A. Multi-level partitioning and distribution of the assignment problem for large-scale multi-robot task allocation. In: . MIT Press, 2012. Disponível em: <https://doi.org/10.7551/mitpress/9481.003.0029>.

NOGUEIRA, C. M. M. *et al.* Promessas e limites do SiSU: uma análise do preenchimento das vagas em cursos de elite. **Educação e Pesquisa**, SciELO Brasil, v. 43, p. 665–682, 2017. Disponível em: <https://doi.org/10.1590/0102-4698161036>.

OLIVEIRA, T. d. S.; SOUZA, C. P. d. *et al.* SISU allocation with indifference among admission modalities: a MAX-HRT model and comparison of tie-breaking criteria. In: **Proceedings of the LVIII Simpósio Brasileiro de Pesquisa Operacional (SBPO)**. Minas Gerais, MG: SBPO, 2026. Trabalho em avaliação SBPO 2026.

SCOTT, S. **A Study Of Stable Marriage Problems With Ties**. Tese (Doutorado) – University of Glasgow, Glasgow, 2005. Disponível em: <https://www.dcs.gla.ac.uk/~davidm/pubs/SandyScottThesis.pdf>. Acesso em: 27 maio 2026.

WEST, D. B. *et al.* **Introduction to graph theory**. [S. l.]: Prentice hall Upper Saddle River, 2001. v. 2.

WILSON, R. J. **Introduction to graph theory**. [S. l.]: Pearson Education India, 1979.

ZHU, Y.; SEGARRA, S. Hypergraph cuts with edge-dependent vertex weights. **Applied Network Science**, v. 7, p. 1–20, 2022. Disponível em: <https://doi.org/10.1007/s41109-022-00483-x>.